Introduction to JavaScript

Lec-1

Made with GAMMA

What is JavaScript ?

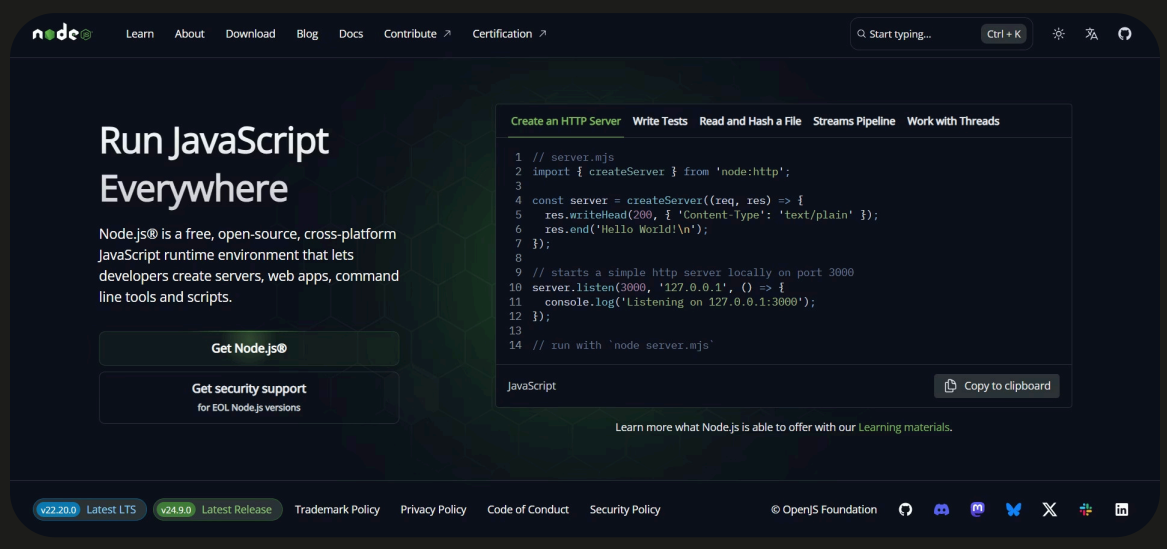
- A **programming language** mainly used for making web pages **interactive and dynamic**.
- Runs in the **browser (client-side)**, but can also run on **servers** with **Node.js (Runtime Environment of JavaScript)**
- Despite the name, **JavaScript is not Java**, They are completely different languages with different use cases.
- Code runs inside a **JavaScript engine** (e.g., V8 in Chrome, SpiderMonkey in Firefox).
- It is **interpreted**, not compiled like C++ or Java.
- Executes **line by line (but can optimize in background)**.
- It is a **Weakly Typed** Language **(No need to declare datatype).**



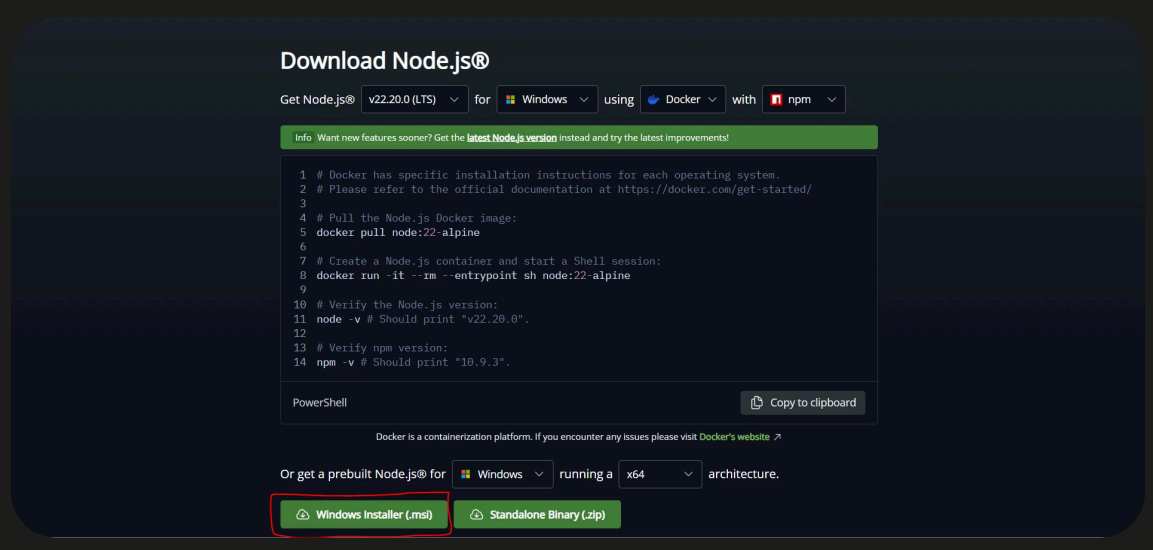
Made with GAMMA

Setting up for JS

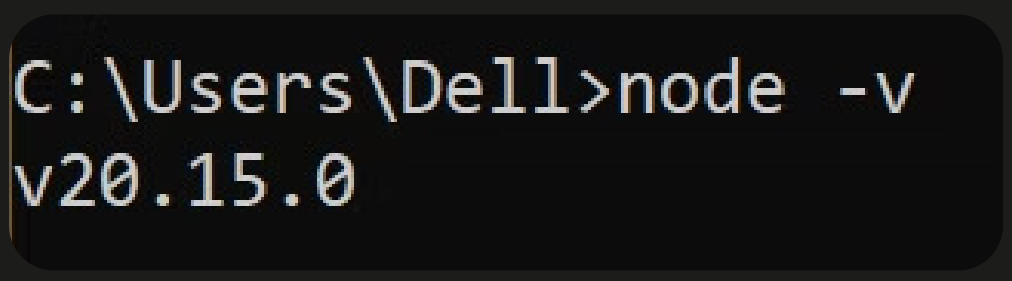
- Installing **NodeJS** is important to run JS, outside our browser.



- Visit the following URL: **https://nodejs.org/en/download**, and click on **Windows Installer (.msl).**



- After Successful installation, open **command prompt** and **run the following command** and match the expected output as shown in the following image.



- If it is showing you **any version of node.js**, that means **Your Installation was Successful.**

Helpful VS Code extension



Made with GAMMA

Linking JavaScript to HTML

- There are **two** primary methods to link JavaScript to HTML.

1. Embedding JavaScript Directly in HTML:

- This method is **suitable for small, simple scripts that are specific to a particular HTML page.**
- Use the `<script>` tags: Place your JavaScript code directly within `<script>` and `</script>` tags in your HTML document.
- **Eg:**

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Intro to JS</title>
</head>
<body>
    <h1>Intro to JS</h1>
    <script>
        // Your JavaScript code goes here
    </script>
</body>
</html>
```

- You can place embedded JavaScript in either the `<head>` or `<body>` section.
- Placing it in the `<head>` **means it will execute before the page content,** while placing it in the `<body>` (ideally just before the closing `</body>` tag) **allows it to interact with the fully loaded HTML.**

2. Linking an External JavaScript File (recd):

- This is the **recommended** and most common approach for larger projects as it promotes code organization and reusability.
- **Create a JavaScript file** Create a new file with a `.js` extension (e.g., `script.js`).
- **Write your JavaScript code** in this file.
- **Link the file in your HTML** in your HTML file, use the `<script>` tag with the `src` attribute to point to the path of your JavaScript file.
- It is generally recommended to **place the `<script>` tag just before the closing `</body>` tag.** This ensures that the **HTML content is loaded and rendered before the JavaScript attempts to interact with it**, preventing potential errors.
- **Eg:**

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Intro to JS</title>
</head>
<body>
    <h1>Intro to JS</h1>
    <!--

    Your HTML Code goes here

    -->
    <!-- Javascript link using script tag and src attribute -->
    <script src="./script.js"></script>
</body>
</html>
```

Made with GAMMA

Hello World in JavaScript

- In JavaScript, anything can be **printed** or **logged** in the following way:



- A console is a panel that displays important messages like errors. If we want to see things by them appearing on our screen, we can print/log to our console directly.
- In JavaScript, the **console** keyword **refers to an object, a collection of data and action/method**, that we can use in our code.
- Keywords are nothing but **Interpreter aware words.**
- One action or method that is built into the console object is the **.log()** method. When we write **console.log()** what we put **inside a parenthesis will get printed or logged to the console.**
- Eg:

```
console.log(5) // Will log 5 to the console
```

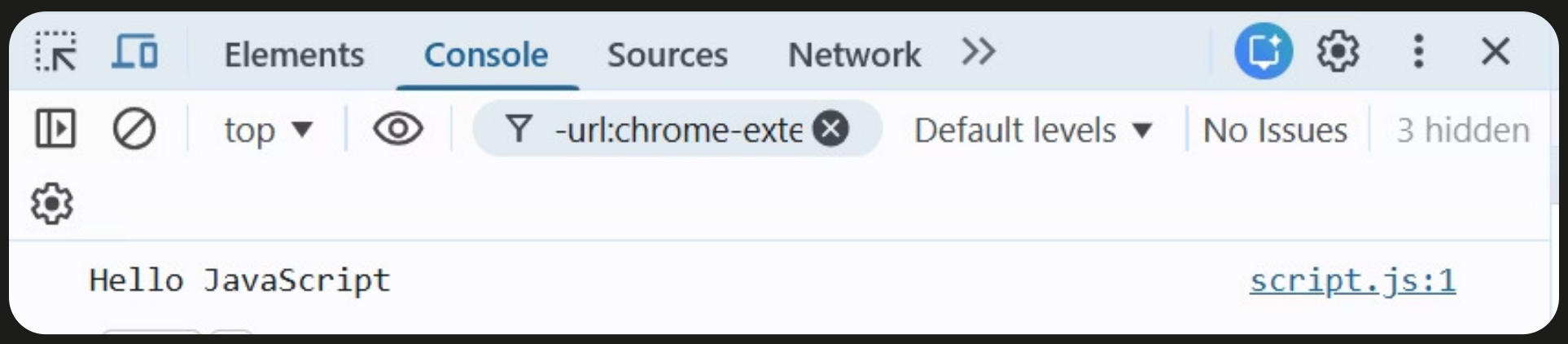
Ways to see output in JavaScript

- **Way-1 :** Open Terminal in VS Code (make sure you open it in same folder in which .js file is) and type the following command.

```
node js_file_name.js
```

> **Note:** after node, you should write your file name, here **script.js** is written considering our existing .js file name is **script.js**.

- **Way-2:** Link your .js file to an .html file creating both of them inside a same folder and open .html file using **live server (VS Code extension)**, after that open the **inspect tool** and **navigate to console tab** and see the output.



Made with GAMMA

Variables in JavaScript

- In programming, a variable is a **container for a value**.
- Variables also provide a way of **labeling data with a descriptive name**, so our programs can be understood more clearly by the reader and by ourselves.

```
a = "John"
console.log(a) // This will print John
a = 25
console.log(a) // This will print 25
```

Above way of defining variables is not recommended in JavaScript

"use strict" directive

- The **"use strict"** directive was new in ECMAScript version 5.
- It defines that JavaScript code should be executed in "strict mode".
- It is not a statement. It is a literal expression, ignored by earlier versions of JavaScript.
- It should **always be declared at the beginning of your JavaScript file**.

```
"use strict" // declared at the top of the Js file
a = "John"
console.log(a)
```

Above lines of code will result in **Uncaught ReferenceError** : a is not defined

Creating a variable using "var" keyword

- **Var,** short for variable is a JS keyword that **creates** or **declares** a new variable.
- **Eg:**

```
var myName = "Mike"
console.log(myName) // This will print Mike
myName = "Micheal"
console.log(myName); // This will print Micheal
```

- If we declare a variable using **var** keyword and **left it unassigned with any value**, it will result as **"undefined"** in the console.
- **Eg:**

```
var unAss
console.log(unAss) // This will print undefined
```

Creating a variable using "let" keyword

- The **let** keyword signals that the **variable can be assigned a different value**.
- **Eg:**

```
let meal = "BigMac"
console.log(meal) // This will print BigMac
meal = "Burrito"
console.log(meal) // This will now print Burrito
```

- Similar to var, if we create a a variable using **let** keyword and **left it unassigned with any value**, it will result as **"undefined"** in the console.
- **Eg:**

```
let unAsslet
console.log(unAsslet); // This will print undefined
```

Creating a variable using "const" keyword

- Variables are declared using **const** in a same way as we declare **let** and **var.**
- **Eg:**

```
const constVar = "Julie"
console.log(constVar); // This will print Julie
```

- Using **const**, it is **not possible to declare a variable without assigning a value** to it.
- **Eg:**

```
const unAssconst // This will result in error - 'const' declaration must be initialised.
```

- Using **const**, it is also **not possible to reassign a new value to a variable which is already been declared and assigned** some value.
- **Eg:**

```
const constVar = "Julie"
constVar = "Mia"
console.log(constVar);
```

Above lines of code will result in **Uncaught TypeError**: Assignment to constant variable.

Note: What to use when ?

- While trying to decide between which keyword to choose while declaring your variable, remember:
- **use "let":** If you wish to reassign it's value later on
- **use "const":** If you do not wish to reassign a new value letter on
- Avoid using **"var"** due to issue of **block scope**.

Made with GAMMA

Datatypes

- Datatypes are **used to indicate the type of data** that we use in programming.
- Datatypes in JS are classified into 2 types based on- **how they're stored in a memory** and **whether they're of fixed size or not**, the types are as follows:
- **Primitve Datatypes:** These are of **fixed size** and they're **directly stored into stack memory**.
- **Non-Primitive Datatypes:** These are not of **fixed size** and their **data is stored in heap memory** and **reference is stored in stack memory**.

Primitive Datatypes

1. **number:** These includes any number, including number with decimals.

Eg:

```
let num1 = 25
console.log(num1); // This will print 25
num1 = 3.14
console.log(num1); // This will print 3.14
let num2 = 27.63
console.log(num2); // This will print 27.63
```

Note: Not-a-Number (NaN)

- In JavaScript **NaN** is a **short form** for- **"Not-a-Number"**.
- NaN is a number which is not **legal number**.
- **Eg:**

```
let notanum = NaN
console.log(notanum); // This will print NaN
console.log(typeof notanum); // This will print number
```

2. **string:** Any **Character** or **grouping of character**, surrounded by **single quotes (' ')** or **double quotes (" ").**

Eg:

```
let str1 = 'Hello'
console.log(str1); // This will print Hello
str1 = "World"
console.log(str1); // This will print World
```

Note: typeof operator

- We can identify type of any value using **typeof** operator.

```
let sillyString = "23"
console.log(sillyString); // This will print 23

// In order to know 23 which is getting printed in console is number or string
console.log(typeof sillyString); // This will print string
```

3. **bigInt**: Any number lying in the **range** of **$-(2^{53}-1)$** to **$(2^{53}-1)$, while declaring bigInt we append 'n' to that number.**

Eg:

```
let bigIntvar = 124676342n
console.log(bigIntvar); // This will print 124676342n
```

4. **boolean:** This datatype **only has two possible values**- either **true** or **false.**

Eg:

```
let boolvar = true
console.log(boolvar); // This will print true
console.log(typeof boolvar); // This will print boolean
boolvar = false
console.log(boolvar); // This will print false
```

5. **null:** This represents the **intentional absence of a value** and is represented by the **keyword null.** It's type is **object.**

Eg:

```
let nullvar = null
console.log(nullvar); // This will print null
console.log(typeof nullvar); // This will print object
```

6. **undefined:** It is also **used to represent the absence of a value** though it has **different use than null**. Undefined means that **the given value does not exists**.

Eg:

```
let undefVar
console.log(undefVar); // This prints undefined because the value to that variable does not exists.
```

7. **symbol:** Symbols are unique identifiers, useful in more complex coding.

Eg:

```
let sym1 = Symbol('123')
console.log(sym1); // This will print Symbol(123)
let sym2 = Symbol('123')
console.log(sym2); // This will print Symbol(123)
console.log(sym1 === sym2); // This will print false
```

Made with GAMMA

Non Primitive Datatypes

1. **arrays** : An Array is an object type designed for storing data collections. It's type is **Object**

Eg:

```
let arr1 = ['Car','Bike','Cycle']

console.log(arr1); // This will print ['Car', 'Bike', 'Cycle']
console.log(typeof arr1); // Object
```

2. **functions:** Functions in JavaScript are **reusable blocks of code** designed to perform specific tasks.

Eg:

```
function greet(){
    console.log("Greetings given");
}
greet()
```

Output: Greetings given

3. **Objects:** An **Object** is a variable **that can hold many variables**. They are collections of **key-value pairs.** It's type is **Object.**

Eg:

```
let car = {
    brand: "Volkswagon",
    mileage: "20kmpl",
    color: "grey",
    model: 'F50',
    carRun()
    {
        console.log("Car Ran !!");
    },
}
console.log(car); // This will print {brand: 'Volkswagon', mileage: '20kmpl', color: 'grey', model: 'F50'}
console.log(typeof car); // This will print Object
console.log(car.color); // This will print grey
car.carRun() // This will invoke carRun method inside car object
```

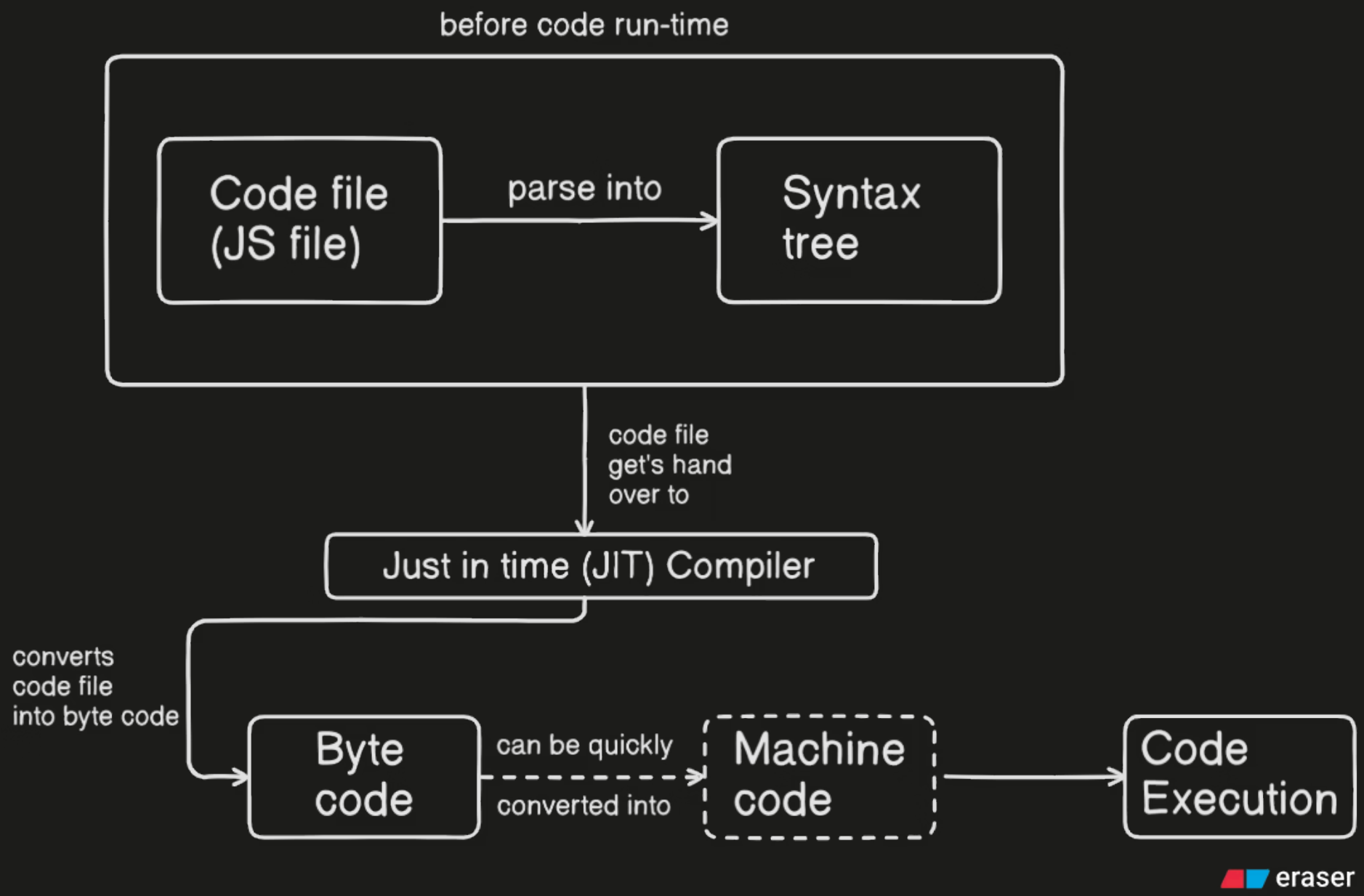
Made with GAMMA

Basics of JavaScript

Lec-2

Made with GAMMA

How JavaScript Executes behind the scene



- Initially your **Js code file** get's parsed into a **syntax tree** by **node or browser's V8 engine.**
- The **role of syntax Tree** is to **check and verify syntax of your code.**
- After that the **code file get's handed over** to **Just In Time (JIT) compiler (JIT is an inbuilt part of Node js)**.
- **JIT converts code file into byte code.**
- **If required byte code** can **convert** code file into **machine code**.
- Then the code finally get's **executed**.

Made with GAMMA

Operators in JavaScript

- In **JavaScript** there are following types of operators:

1. **Arithmetic Operators**
2. **Assignment Operators**
3. **Comparison Operators**
4. **Logical Operators**

Arithmetic Operators

- They're used to **perform arithmetic operations on numbers**.

Operator	Name	Example
+	Addition	x + y
-	Subtraction	x - y
*	Multiplication	x * y
**	Exponentiation	x **y
/	Division	x / y
%	Modulus	x % y
++	Increment	x ++
- -	Decrement	x - -

- **Eg:**

```
let var1 = 20
let var2 = 50

console.log(var1 + var2)
70

console.log(var1 - var2)
-30

console.log(var1 * var2)
1000

let var3 = 5
let var4 = 2

console.log(var3 ** var4)
25

console.log(var3 / var4)
2.5

console.log(var3 % var4)
1

console.log(var3++)
6

console.log(var4 --)
1
```

Made with GAMMA

Assignment Operators

- Assignment operators **assign values to JavaScript variables**.

Operator	Example	Same as
=	x = y	x = y
+ =	x += y	x = x + y
- =	x -= y	x = x - y
* =	x *= y	x = x * y
** =	x **= y	x = x ** y
/ =	x /= y	x = x / y
% =	x %= y	x = x % y

- **Eg:**

```
let x = 10
let y = 5

console.log(x)
10
console.log(y)
5
console.log(x+=y)
15
console.log(x-=y)
10
console.log(x*=y)
50
console.log(x/=y)
10
console.log(x%=y)
0
```

Comparison Operators

- Comparison operators are used to **compare two values**.
- Comparison operators always return **true** or **false.**

Operator	Description	Example
= =	equal to	x == 5
= = =	equal value and equal type	x === 5
! =	not equal	x != 5
! = =	not equal value or not equal type	x !== 5
>	greater than	x > 5
<	less than	x < 5
> =	greater than or equal to	x >= 5
< =	less than or equal to	x <= 5

- **Eg:**

```
let var1 = 'hello'
let var2 = 'hi'

console.log(var1 == var2)
false

console.log(var1 === var2)
false

var2 = 'hello'

console.log(var1 === var2)
true

console.log(var1 != var2)
false

console.log(var1 !== var2)
false

let num1 = 15
let num2 = 13

console.log(num2 > num1)
false

console.log(num2 < num1)
true

num1 = 13

console.log(num2 >= num1)
true

console.log(num2 <= num1)
true
```

Made with GAMMA

Logical Operators

- Logical Operators are used to **combine Boolean expressions**.
- They can be used to modify results of comparison.

Operator	Name	Example
&&	**AND**	(x < 10 && y > 1) is true
||	**OR**	(x === 5 || y === 5) is false
!	**NOT**	!(x === y) is true

Eg:

```
let var1 = 50;
let var2 = 100;

console.log(var1 < var2 && var1 != var2)
true
console.log(var1 < var2 && var1 !== var2)
true

var2 = 'hundred';
console.log(var1 < var2 && var1 != var2)
false
console.log(var1 < var2 && var1 !== var2)
false

var2 = 100;
console.log(var1 < var2 || var1 != var2)
true
```

```
let s = 17
let k = 17
console.log(!(s === k))
false
```

Null Coalescing Operator (??)

- The `??` operator returns the right operand when the left operand is **null** or **undefined.**
- **Otherwise it returns the left operand**.

Eg:

```
let test1 = 'testVar1'
let test2 = 'testVar2'

let result = test1 ?? test2
console.log(result)
testVar1

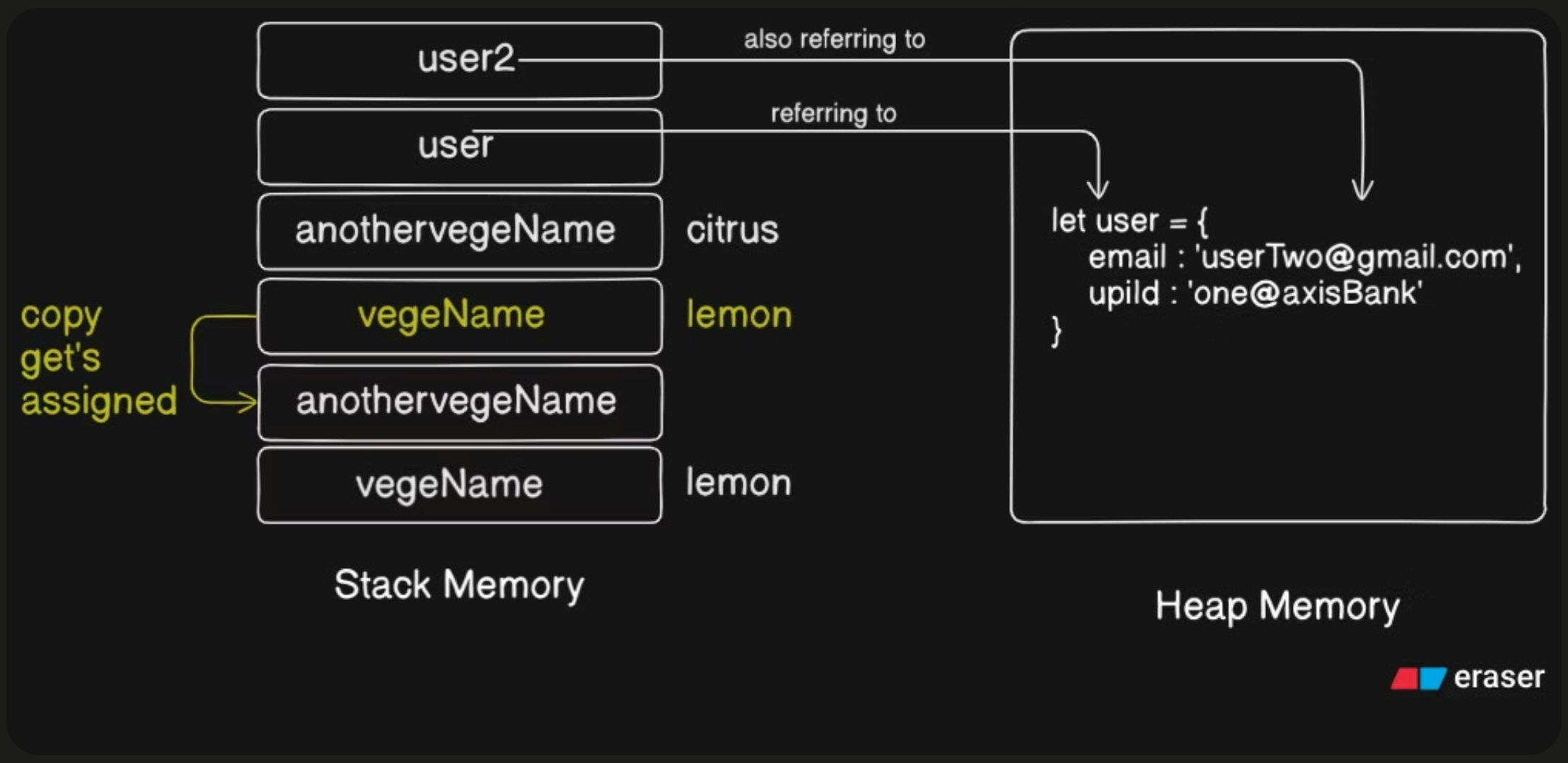
test1 = null
result = test1 ?? test2
console.log(result)
testVar2


test1 = undefined
result = test1 ?? test2
console.log(result)
testVar2
```

Made with GAMMA

Stack Memory and Heap Memory

Stack Memory	Heap Memory
Only **primitive datatypes** are stored in **stack memory.**	Only **non-primitive datatypes** are stored in **heap memory.**
When any data is stored into **stack memory** we get **copy of that data.**	When any data is stored into **heap memory** we get **reference of that data**



Eg:

```
"use strict"
let vegeName = 'lemon'
let anothervegeName = vegeName

console.log(anothervegeName);

anothervegeName = 'citrus'

console.log(anothervegeName);

console.log(vegeName);


let user = {
    email : 'userOne@gmail.com',
    upiId : 'one@axisBank'
}

let user2 = user
user2.email = 'userTwo@gmail.com'

console.log(user2.email);
console.log(user.email);
```

Made with GAMMA

Strings and Numbers in JS

Lec-3

Made with GAMMA

Strings in JS

- Strings are for storing text.
- Strings are written within single quotes or double quotes.
- **Eg:**

```
let carName = "Ferrari 250";
let carName = 'Ferrari 250';
```

Note:

In order to find length of any string we use **length property** along with the use of **'dot(.) operator'**.

Eg:

```
let name = 'Jimmy'
console.log(name.length) // Output: 5
```

String Concatenation

- String concatenation is a **way to join one string with another string or to join a string with data of some other type**.
- **Eg:**

```
let age = 20
let stmt = "My age is: "
console.log(stmt + age); // output: My age is 20
```

Note:

Concatenating Strings with numbers or with any other datatype in such way is not a good practise in writing your JS code.

Template Literals

- Template literals use **backticks (` `)** to define a string rather than **single/double quotes (' ' / " ")**.
- **Eg:**

```
let instrument = `Guitar`;
```

String Interpolation

- **Template Strings** allow variables in strings.
- Template strings provide an easy way to **interpolate** variables in strings.
- **Eg:**

```
let currentAge = 16
console.log(`My Current age is: ${currentAge}`); // output: My current age is 16
```

Made with GAMMA

String Methods

- String Methods are used to **perform manipulation with Strings**.
- Since strings in JS are **primitive** and **mutable** all the string methods whenever make any changes, produces a new string.
- These methods **never change the original string**.

toUpperCase() : Converts a String into an upper case.

```
let gameName = 'thanos'
console.log(gameName.toUpperCase()); // output: THANOS
```

toLowerCase() : Converts a String into an lower case.

```
let gameName = 'THANOS'
console.log(gameName.toLowerCase()); // output: thanos
```

charAt() : returns the character at a specified index (position) in a string.

```
let gameName = 'thanos'
console.log(gameName.charAt('2')); // output: a
```

indexOf() : Searches a string for a specific value and returns the index (position) of its first occurrence.

```
let gameName = 'thanos'
console.log(gameName.indexOf('n')); // output: 3
```

slice() : Extracts a part of a string and returns the extracted part in a new string. The method takes 2 parameters: start position, and end position **(end not included).**

```
let gameName = 'thanos'
console.log(gameName.slice(0,4)); // output : than
```

Note:

slice() allows negative values as parameters. If a parameter is negative, the position is counted from the end of the string.

```
let gameName = 'thanos'
console.log(gameName.slice(0,-4)); // output : th
```

substring() : This method is similar to **slice(),** the difference is that negative values as parameters are ignored by this method and even after giving negative parameters it starts from $0th$ position only.

```
let gameName = 'thanos'
console.log(gameName.substring(1,4)); // output : han
```

trim() : removes whitespace from both sides of a string.

```
let anotherName = "   thanos   "
console.log(anotherName);  // output:    thanos
console.log(anotherName.trim()); // output: thanos
```

replace() : replaces a specified value with another value in a string. This method by default replaces **only the first** match. This method is also a **case sensitive** method.

```
let RandomSentence = 'Use Claude, Gemini or Gemini'
console.log(RandomSentence.replace('Gemini', 'ChatGPT'));
// output: Use Claude, ChatGPT or Gemini
```

Note:

In order to replace all matches we use a **regular expression** with **'/g' flag.**

```
let RandomSentence = 'Use Claude, Gemini or Gemini'
console.log(RandomSentence.replace(/Gemini/g, 'ChatGPT'));
// output: Use Claude, ChatGPT or ChatGPT
```

In order to replace all matches **irrespective of the case** we use a regular expression with **'/i' flag.**

```
let ideTool = 'I Love ANTIGRAVITY'
console.log(ideTool);
// output: I Love ANTIGRAVITY
console.log(ideTool.replace(/antigravity/i,'VSCode'));
// output: I Love VSCode
```

includes() : This method is used to **determine whether a string contains a specific substring**. It returns a **boolean value (true or false)**.

```
let url = "https://globalSearch.com/Search"
console.log(url.includes('global')); // Output: true
```

Made with GAMMA

Numbers in JS

- JavaScript has **only one type** of number.
- Numbers can be written **with** or **without decimals**.
- JavaScript numbers are always **64-bit** floating point.

Number Methods in JS

toString() : Returns a number as a string.

```
const aNum = 65;
console.log(aNum);  // Output: 65
console.log(typeof aNum);  // Output: number

console.log(typeof aNum.toString()); // Output: string
```

Note:

toString() method can also take a optional argument to convert a number to a different base.

```
let aNum = 15
console.log(aNum.toString(2)) // Output: 1111
```

toFixed() : Returns a number as a string, with the number written with a specified number of decimals.

```
const aNum = 3.141592

console.log(aNum.toFixed(2)) // Output: 3.14

console.log(aNum.toFixed(5)) // Output: 3.14159

console.log(typeof aNum) // Output: number

console.log(typeof aNum.toFixed(3)) // Output: string
```

toPrecision() : Returns a number as a string, with a number written with a specified length.

```
let aNum = 9.56563
console.log(aNum.toPrecision(2)) // Output: 9.6
console.log(aNum.toPrecision(3)) // Output: 9.57
console.log(aNum.toPrecision(4)) // Output: 9.566
console.log(aNum.toPrecision(10)) // Output: 9.565630000
```

Note:

toFixed() starts **counting after the decimal**, whereas **toPrecision() counts all meaningful digits** in entire number.

toLocaleString() : This is the primary method for making data human-readable based on a user's language and region.

```
const aAmnt = 744000

console.log(aAmnt.toLocaleString()) // Ouput: 744,000

console.log(aAmnt.toLocaleString('en-IN')) // Output: 7,44,000
```

Made with GAMMA

JavaScript Math Object

- The JavaScript Math object allows you to perform mathematical tasks
- The Math object is static.
- The Syntax to Math any number is:

```
Math.method(number)
```

.round() : Returns a nearest integer.

```
const numb = 69.14
console.log(Math.round(numb)) // Output: 69
```

.ciel() : Returns a rounded up nearest integer.

```
const numb = 69.14
console.log(Math.ceil(numb)) // Output: 70
```

.floor() : Returns a rounded down nearest integer.

```
const numb = 69.14
console.log(Math.floor(numb)) // Output: 69
```

.pow(x,y) : Returns the value of x to the power of y.

```
console.log(Math.pow(2,4)) // Output: 16
```

.sqrt() : Returns the square root of a value.

```
console.log(Math.sqrt(144)) // Output: 12
```

.abs() : Always returns the absolute (positive) value.

```
console.log(Math.abs(-63)) // Output: 63
```

.min() : Used to find lowest value in the list of arguments.

```
console.log(Math.min(53,85,96,2,15)) // Output: 2
```

.max() : Used to find highest value in the list of arguments.

```
console.log(Math.max(53,85,96,2,15)) // Output: 96
```

.random() : returns a random number between 0 (inclusive), and 1 (exclusive). It always returns a number lower than 1.

```
console.log(Math.random()) // Output: 0.8923472168130379
```

Note:

Suppose we have to generate a **random number between 1 to 10.**

```
console.log(Math.round(Math.random()*10)+1)
```

Suppose we have to generate a **random number between a given range.**

```
const min = 25
const max = 50

console.log(Math.round(Math.random() * (max - min + 1) + min));
```

Made with GAMMA

Conditionals in JS

Lec-4

Made with GAMMA

If Statement

- The **If Statement** in JS executes the code only when a condition is **true**.
- **Eg :**

```
let cartEmpty = true

if (cartEmpty){
    console.log('Your Cart is Empty');
}

// Output: Your Cart is Empty.
```

If you've to print single statement after passing it through a 'if' statement, alternatively it can be done as follows:

```
let cartEmpty = true
if (cartEmpty) console.log("Your Cart is Empty");

// Output: Your Cart is Empty.
```

```
let randomNum = 56
if (randomNum > 100){
    console.log("The number is greater");
}
// Nothing will get printed to console as the condition given in if statement her is turning out to be false.
```

Truthy Value and Falsy Value

Truthy	**true, "false", "0", 1 , [], {}**
Falsy	**false , 0, -0, "", null, NaN, undefined**

Made with GAMMA

Else Statement

- The Else Statement is to specify a block of code to be executed **only when a if condition turns out to be false**.
- **Eg :**

```
let moneyPresent = false
if(moneyPresent){
    console.log("Account balance");
}
else{
    console.log("You don't have any balance in your account");
}

//Output: You don't have any balance in your account
```

Solve This Practice Question

Write a simple JavaScript program to check whether the person is **eligible to drive or not**.

Create 2 variables to store a person's **Age** and the **Country** which he's living in

If the person's **age is greater than or equal to 16** and **country is USA** , print ***"You're eligible to drive !!"***

else print ***"You're not eligible to drive !!"***

Made with GAMMA

Ternary Operator

- Ternary Operator is a **shorthand** for writing **if-else conditional statements**.
- It is called **Ternary Operator** because it **takes three operands**.
- **Eg :**

Traditional if-else conditional

```
let health = 'Sick'
if(health.toLowerCase() === 'sick'){
    console.log('Go to doctor');
}
else{
    console.log("Enjoy at home");
}
```

Ternary Operator

```
console.log((health.toLowerCase() === 'sick') ? "Go to doctor" : "Enjoy at home");
```

Made with GAMMA

Else if statement

- The Else if Statement is to specify a **new condition** if the **first condition is false**.
- **Eg :**

```
let foodItem = 'Hello'

if (foodItem.toLowerCase() === 'breakfast'){
    console.log("Idli Sambhar");
}
else if(foodItem.toLowerCase() === 'lunch'){
    console.log('Non veg Thali');
}
else if(foodItem.toLowerCase() === 'dinner'){
    console.log('Veg Thali');
}
else{
    console.log("Invalid input");
}
```

Solve This Practice Question

Imagine a home automation tool which automatically turns on **Heater, Fan** or **Air Conditioner** based on sensing the current temperature of the house.

The conditions to be followed are:

- If current temperature is below **18°C,** then **"Turn the Heater on"**
- If current temperature is between **18°C and 25°C (inclusive,** then **"Turn the Fan on"**
- If current temperature is **above 25°C,** then **"Turn the AC on"**

Made with GAMMA

Switch Statement

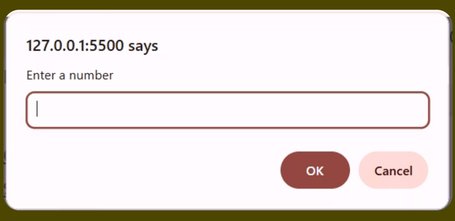
- Based on a condition, **switch** statement **selects one or more code blocks to be executed**.
- switch executes an code block **that matches an expression**.
- Switch acts as an **alternative to many if-else statements** hampering readability of the code.
- In switch statement we've to give an **break** keyword for **preventing a "fall-through"**.
- The **default** keyword specifies a block of code to run **if there is no case match**.
- **Without** the **break keyword**, the code will **keep executing the next case block including the default case block,** even when **values do not match** with the expression.

Note:

How to Accept user input in JS

- **prompt() :** This method displays a dialog box that prompts the user for input.
- This method **returns the input value** if the user clicks **"OK"**, otherwise it returns **null**
- **Eg :**

```
prompt("Enter a number")
```



- prompt() takes an input from user and **always store it in form of String**.

Methods to convert Strings into number

- **Number() :** Returns a number, converted from its argument.

```
let Stringnum = "100"
console.log(Stringnum); // 100
console.log(typeof Stringnum); // string
console.log(Number(Stringnum)); // 100
console.log(typeof Number(Stringnum)); // number
```

- **parseFloat() :** Parses a string and returns a floating point number.

```
let Stringnum = "69.99"
console.log(Stringnum); // 69.99
console.log(typeof Stringnum); // string
console.log(parseFloat(Stringnum)); // 69.99
console.log(typeof Number(Stringnum)); // number
```

- **parseInt() :** Parses a string and returns an integer.

```
let Stringnum = "69.99"
console.log(Stringnum); // 69.99
console.log(typeof Stringnum); // string
console.log(parseFloat(Stringnum)); // 69.99
console.log(typeof Number(Stringnum)); // number
```

- **Example for switch case :**

```
let Daynum = parseFloat(prompt("Enter a number"))
console.log(Daynum);

switch(Daynum){
  case 1:
    console.log("It's Sunday");
    break;
  case 2:
    console.log("It's Monday");
    break;
  case 3:
    console.log("It's Tuesday");
    break;
  case 4:
    console.log("It's Wednesday");
    break;
  case 5:
    console.log("It's Thursday");
    break;
  case 6:
    console.log("It's Friday");
    break;
  case 7:
    console.log("It's Saturday");
    break;
  default:
    console.log("Invalid input");
}
```

Solve This Practice Question

Imagine you're builiding a program for an tele caller of an bank application which does different tasks on basis of selection of different option by the user.

- Initially, print message to the user **"Welcome to the BANK OF INDIA"**
- Then print the message in following way:

  "Press 1 to know your account balance"

  "Press 2 to edit your account details"

  "Press 3 to verify your EKYC"

  "Press 4 to explore our loan options"

- Take input from user using **prompt()**
- Manipulate the input using switch case in following manner

If user inputs **1** print **"You've selected to view your account balance"**

If user inputs **2** print **"You've selected to edit your account details"**

If user inputs **3** print **"You've selected to verify your EKYC"**

If user inputs **4** print **"You've selected to explore our loan options"**

Made with GAMMA

Solve these Questions on Conditionals

Question 1 — Form validator for a signup page

Imagine you're building the frontend of a user registration page. Before a new user is allowed to create their account, your JavaScript code needs to validate their details on the spot.

Create the following variables to represent the user's submitted data:

- A variable **username** storing a string value
- A variable **age** storing a number
- A boolean variable **agreedToTerms**

Now validate the data using conditionals in the following way:

- Using an **if statement**, check if **username** is **non-empty** — if it is **empty (falsy)**, print **"Username cannot be empty"**
- Using **if / else if / else**, check the **age**

— if age is **below 13**, **print "Too young to register"**

— if age is **between 13 and 17**, **print "Parent approval required"**

— if age is **18 or above**, **print "Welcome aboard!"**

- Use a **ternary operator** to print **"Terms accepted"** if **agreedToTerms is true**, otherwise **print "Must accept terms to continue"**

Test your code by changing the values of all three variables and observe how the output changes each time.

Question 2 — Delivery status tracker

Imagine you're building the **"Track My Order"** screen of an e-commerce app. The backend sends a numeric status code, and your job is to translate that code into a clear, human-readable message for the customer.

Use pro**mpt()** to ask the user: **"Enter your delivery status code (1–5):"**

Write a **switch statement** that maps each code to a message in the following way:

- If the user inputs **1**, **print "Order placed"**
- If the user inputs **2, print "Packed and dispatched"**
- If the user inputs **3, print "Out for delivery"**
- If the user inputs **4, print "Delivered successfully"**
- If the user inputs **5, print "Delivery failed — contact support"**
- For any **other input**, **print "Invalid status code — please check your order ID"**

Question 3 — Subscription plan upgrade prompt

Imagine you're building the **billing logic for a SaaS dashboard**. Based on the user's current subscription plan and their monthly usage, your program should decide whether to show them an upgrade prompt.

Use **prompt()** to ask: **"Enter your current plan (free / pro / enterprise):"**

Use **prompt() again** to ask: **"Enter your monthly usage in GB:"**

Write an if / **else if / else chain** to handle the following conditions:

- If the plan is **"free"** and usage is **above 2 GB**, print **"Upgrade to Pro for more storage"**
- If the plan is **"pro"** and usage is **above 50 GB,** print "**Consider upgrading to Enterprise"**
- If the plan is **"enterprise"**, print **"You're on our best plan!"**
- For any **other combination**, print **"Usage is within limits, no upgrade needed"**

Additionally, create a **boolean** variable **isBillingActive** and **set it yourself**. Use a ternary operator to print either **"Billing: Active"** or **"Billing: Inactive"** based on its value.

Test the full program by trying different combinations of plan, usage, and billing status.

Made with GAMMA

Arrays in JS

Lec-5

Made with GAMMA

Arrays in JS

- An Array is an **object type** designed for **storing data collections**.
- An array can **hold many values under a single name**, and you can **access the values by referring to an index number**.
- Key **Characteristics of an Array** are as follows:

1. **Zero-Indexed**: The **first element** is always at **index 0**, the **second at 1**, and so on.
2. **Ordered Collection**: Elements are stored in a specific sequence, and that order is maintained based on their index.
3. **Dynamic and Resizable**: Unlike some other languages, **JavaScript arrays do not have a fixed size**; they grow or shrink automatically as you add or remove elements.
4. **Heterogeneous (Mixed Types)**: A single array can hold elements of different data types simultaneously, such as a mix of strings, numbers, objects, and even other arrays.
5. **Reference Type**: Arrays are technically **objects**. The **typeof** operator returns **object** for an array, and they are copied by reference rather than by value.

- It is always an best practice to declare an Array using **const** keyword.
- Generally there are two ways of declaring an array:

Way-1 : Using Array literal.

Way-2 : Using 'new' keyword.

Using Array literal

```
const cars = ['Volvo','BMW','Audi','Hyndai']
```

Using 'new' keyword

```
const cars2 = new Array('Volvo','BMW','Audi','Hyndai')
```

- In most cases we always prefer **declaring an array by creating an Array literal (Way-1)**.
- We can **access an array element** by referring to the **index number.**
- **Eg :**

```
const sports = ['football','cricket','basketball','golf','rugby']
console.log(sports[3]); // golf
console.log(sports[1]); // cricket
```

- We can also create an empty array, and provide elements later.
- **Eg :**

```
const groceries = []
groceries[0] = "Egg"
groceries[1] = "Milk"
groceries[2] = "Sugar"
groceries[3] = "bread"
console.log(groceries); //['Egg', 'Milk', 'Sugar', 'bread']
```

- We can any existing element of an existing array in the following way:

```
const stationary = ['Pencil','Pen','Eraser','Exam Pad']
console.log(stationary); //['Pencil', 'Pen', 'Eraser', 'Exam Pad']

stationary[2] = 'Sharpner'
console.log(stationary); // ['Pencil', 'Pen', 'Sharpner', 'Exam Pad']
```

- We can find length of any array using the **.length** property.

```
const kitchenEquip = ['knife','spatula','stirrer','mixer']
console.log(kitchenEquip.length); // 4
```

Made with GAMMA

Array Methods

- **.push()** : Adds a new element **at the end** of an array.

```
let fruits = ['kiwi','strawberry','watermelon']
console.log(fruits);
// output: kiwi, strawberry, watermelon
fruits.push('muskmelon')
console.log(fruits);
// output: kiwi, strawberry, watermelon, muskmelon
```

What if we merge two arrays into single array by using .push() method ?

```
const americanCars = ['Ford','Tesla','GM']
const germanCars = ['Volkswagon','BMV','Audi']

americanCars.push(germanCars)
console.log(americanCars);
// output: ['Ford', 'Tesla', 'GM', Array(3)]
```

If we observe above example carefully we can see that, .push() methods considers the 2nd array as one single element and adds up with the elements of 1st array. To overcome this we can use **.concat()** or **spread operator**.

- **.concat()** : Creates a new array by merging (concatenating) the existing arrays.

```
const americanCars = ['Ford','Tesla','GM']
const germanCars = ['Volkswagon','BMV','Audi']

const mergedArrays = americanCars.concat(germanCars)
console.log(mergedArrays);
// output: ['Ford', 'Tesla', 'GM', 'Volkswagon', 'BMV', 'Audi']
```

Alternatively, we can also use **Spread Operator.**

Spread Operator (...)

- This operator splits iterables into individual elements.
- **Eg-1** :

```
let string1 = 'Enginner'
console.log(...string1); // output: E n g i n e e r
```

- **Eg-2 :**

```
const americanCars = ['Ford','Tesla','GM']
const germanCars = ['Volkswagon','BMV','Audi']

const mergedArrays = [...americanCars,...germanCars]
console.log(mergedArrays);
// output: ['Ford', 'Tesla', 'GM', 'Volkswagon', 'BMV', 'Audi']
```

- **Eg-3** :

```
const domesticAnimals = ['dog','cat']
const farmAnimals = ['cow','pig','chickens','goat']

const animals = ['lion','zebra', ...domesticAnimals, 'cheetah','panther', ...farmAnimals]
console.log(animals);
// output: ['lion', 'zebra', 'dog', 'cat', 'cheetah', 'panther', 'cow', 'pig', 'chickens', 'goat']
```

- **pop()** : This method **removes the last element** from an array.

```
const animals = ['lion','zebra', ...domesticAnimals, 'cheetah','panther', ...farmAnimals]
console.log(animals);
// output: ['lion', 'zebra', 'dog', 'cat', 'cheetah', 'panther', 'cow', 'pig', 'chickens', 'goat']
animals.pop()
console.log(animals);
// output: ['lion', 'zebra', 'dog', 'cat', 'cheetah', 'panther', 'cow', 'pig', 'chickens']
```

Made with GAMMA

Array Methods

- **unshift()** : Adds one or more elements to the **beginning** of an array and returns the new length.

```
let cars = ['BMW', 'Audi'];
cars.unshift('Ford');
// ['Ford', 'BMW', 'Audi']
```

- **shift()** : Removes the **first** element from an array and returns that element.

```
let pets = ['Cat', 'Dog', 'Bird'];
let first = pets.shift(); // 'Cat'
// pets is now ['Dog', 'Bird']
```

- **includes()** : This method is a straightforward way to **check if an array contains a specific element**. It returns a **boolean result**. It is an **case sensitive** method.

```
const animals = ['Lion', 'Tiger', 'Bear'];
console.log(animals.includes('Lion')); // true
```

Made with GAMMA

Array Methods

- **indexOf()** : Returns the **first index** at which a given element can be found; returns **-1** if it is not present.

```
let tools = ['Hammer', 'Screwdriver', 'Hammer'];
console.log(tools.indexOf('Hammer')); // 0
```

- **join()** :Concatenates all elements of an array into a **string**, separated by a specified string (comma by default).

```
let letters = ['J', 'S'];
console.log(letters.join('-')); // "J-S"
```

- **slice()** :Returns a **shallow copy** of a portion of an array. It **does not change** the original array..

```
let nums = [10, 20, 30, 40];
let middle = nums.slice(1, 3); // [20, 30] (index 1 to 2)
```

- **splice()** : **Changes** the contents of an array by removing or replacing existing elements and/or adding new ones

```
let colors = ['Red', 'Green', 'Blue'];
colors.splice(1, 1, 'Yellow'); // At index 1, remove 1, add 'Yellow'
// colors is now ['Red', 'Yellow', 'Blue']
```

Made with GAMMA

Array Methods

- **flat()** : Creates a **new array with all sub-array elements concatenated into it** recursively up to the specified depth

```
let nested = [1, [2, [3]]];
console.log(nested.flat(1)); // [1, 2, [3]]
console.log(nested.flat(2)); // [1, 2, 3]
```

- **Array.from()** : Creates a **new array from an array-like or iterable object** (like a String or a Set).

```
console.log(Array.from('Hey')); // ['H', 'e', 'y']
```

- **Array.of()** : Creates a **new array with a variable number of arguments, regardless of the number or type of arguments.**

```
console.log(Array.of(7)); // [7] (Note: Array(7) creates an empty array with length 7)
```

Made with GAMMA

Functions in JS

Lec-6

Made with GAMMA

What are Functions ?

- Functions are **reusable blocks of code** designed to **perform a particular task**.
- Functions are only executed when they're **called** or **invoked**.
- Functions often **return a optional value** back to the caller.
- Whenever we use a **return** statement in a function, **code after it does not get executed**.

Why are functions used ?

- They help us follow **D.R.Y (Do not Repeat Your code)** principle and helps us **reuse our code.**
- They help us **organize** our code into **smaller parts**.
- They makes code **easier to read** and **maintain**.
- They help us **use same code** to **produce different results at different times**.

Declaring and calling a Function

- A function can be created or declared using a **function keyword**, **function name** and a **paranthesis ({ })**.
- In order to **run / execute** the function **we have to call it** using **function name** followed by **open close round brackets ( )**.

```
// function declaration
function sayMyName(){
  return `Hisenberg`;
}
// function calling
const call = sayMyName();
console.log(call) // will return 'Hisenberg'.
```



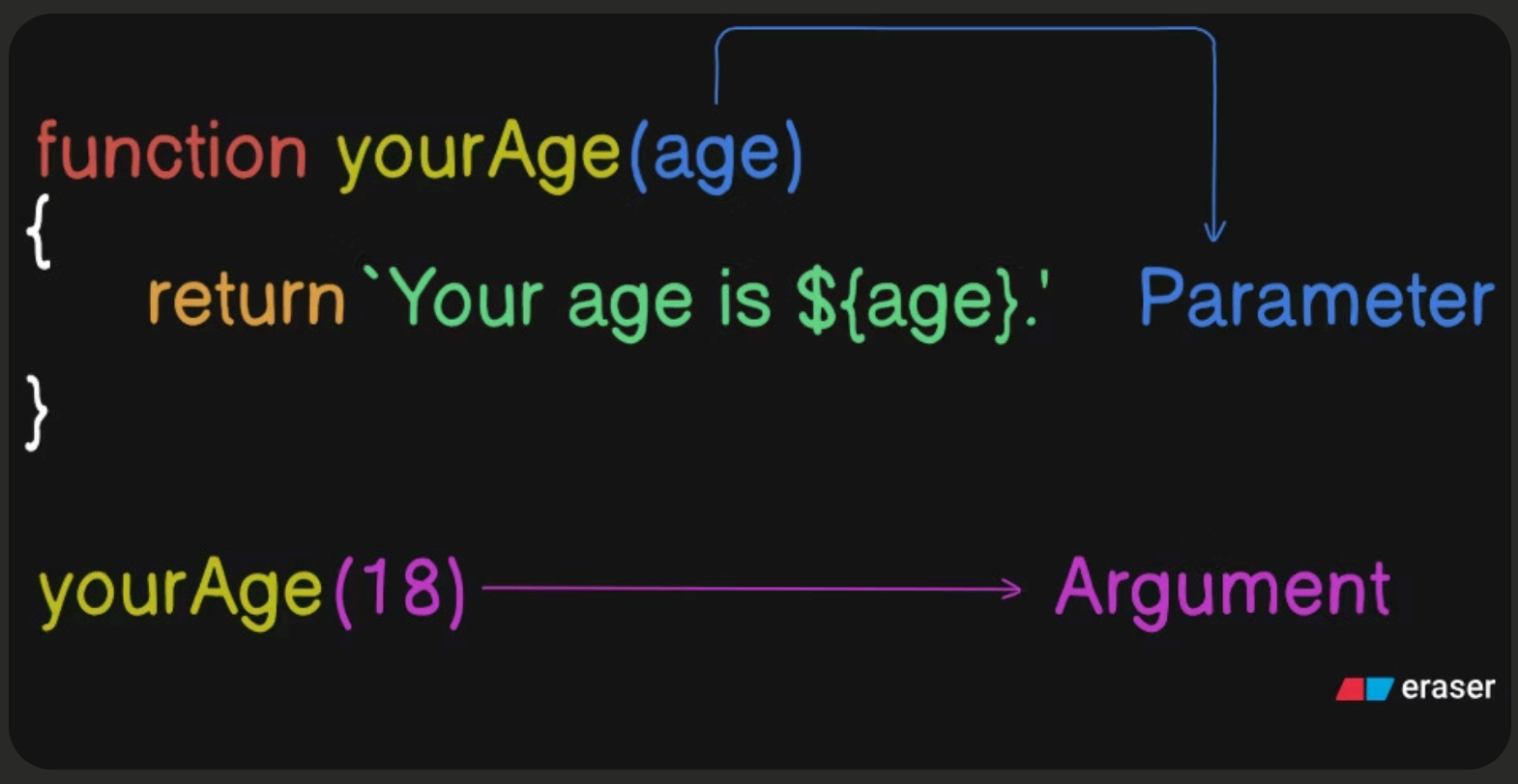
- Suppose, we have just written **sayMyName** instead of **sayMyName()** this meant we would've **just referenced a function** and **not have called it**.

1. **sayMyName** : refers to **function itself**, it **returns the function**.
2. **sayMyName()** : refers to **function result**, it **returns the result**.

Made with GAMMA

Parameters and Arguments in Functions

- **Parameters** : Parameters are the **placeholders** in which values are passed inside the function.
- **Arguments** : Arguments are the **values** which are passed to those placeholders (parameters).



- **Eg :**

```
function iceCreamOrder(order){
    return `You've ordered ${order} ice cream !!`
}
const orderRes = iceCreamOrder('chocolate')
console.log(orderRes); // will print "You've ordered chocolate ice cream !!"
```

- Functions can have **more than one parameters**.
- Arguments are assigned to parameters **in the order as they appear.**
- **Eg** :

```
function foodOrder(order,extra){
    return `You have ordered ${order} with a extra ${extra}.`
}
const orderDetails = foodOrder('burger','slice of cheese')
console.log(orderDetails); // will print "You have ordered burger with a extra slice of cheese."
```

- Functions can also have a **default parameters**.
- **Eg** :

```
function currentAge(birthYear,currentYear = 2026){
    return `Your age is: ${currentYear - birthYear}`
}

const ageDetails = currentAge(2007)
console.log(ageDetails); // will print: "Your age is: 19"
```

Solve This Practice Question

You're building a small feature for a food delivery app. Write a function named **placeOrder** that takes **two parameters**, **foodItem** and **quantity**, and **returns** a string like **"Ordering 2 portions of Biryani"** when called with **"Biryani"** and **2**.

- Call the function with a **food item** and **quantity of your choice**
- Store the returned result in a variable named **myOrder.**

Made with GAMMA

Scope in JS

- Scope determines the **visibility** or **accesibility** of variables.
- JavaScript have 3 types of scopes:

1. **Global Scope**
2. **Function Scope**
3. **Block Scope**

Global Scope

- Variables declared **globally (outside any block or function)** have global scope.
- **Global variables can be accessed from anywhere** in the JavaScript program.
- Variables like **let**, **var** and **const** have same **global scope** when declared outside a block.
- **Eg :**

```
let number1 = 20 // global variable

function additionOfNumber(number2){
    return number1 + number2 // global variable number1 can be accesible in this block as well.
}

console.log(additionOfNumber(40));
```

Function Scope

- All JavaScript functions have their own scope.
- Variables defined **inside** a function are **not accessible (visible) from outside the function**.
- Variables declared with **let**, **var** and **const** are **quite similar** when declared inside a function.
- **Eg** :

```
function randomFunction(){
    let randomVariable = 'random'
}
console.log(randomVariable); // will give Uncaught ReferenceError: randomVariable is not defined
```

Block Scope

- Block Scope was introduced to JavaScript in **ES6**.
- Variables declared with **let** and **const** have **block scope** meaning they're **accessible only within that block**.
- **Eg** :

```
if(true){
    let a = 40
    const b = 60
    var c = 80
}
console.log(a); // Uncaught ReferenceError: a is not defined
console.log(b); // Uncaught ReferenceError: b is not defined
console.log(c); // 80
```

Made with GAMMA

Scope inside a function

- Whenever we define a function inside a another function the **variables declared inside a scope of parent function** can be **accesible inside the scope of child function.**
- But the **variables declared inside the scope of child function cannot** be **accessible in the scope of parent function**.
- **Eg** :

```
function func1()
{
    let var1 = 69

    function func2()
    {
        let var2 = 43
        console.log(var1);
        console.log(var1 + var2);
    }

    func2()
    console.log(var2); // ReferenceError: var2 is not defined at func1
}

func1()
console.log(var1); // ReferenceError: var1 is not defined
```

Anonymous Function

- Functions **without any name** are anonymous functions.
- They are created using **function expression**.
- The **variable** in which they're stored, **it's name is used to call a function**.
- **Eg :**

```
const anonymousVar = function(){
    return `Hi, I am an Anonymous function`
}

console.log(anonymousVar()); // will print "Hi, I am an Anonymous function"
```

Hoisting in JS

- It is a behavior of an interpreter, where **it appears to move declarations of variable**, **functions** and **classes** to the **top of their scope** before the code is executed.

- **Function declaration after defining.**

```
function aFunction(){
    console.log(`Hi, I am a function`);
}
aFunction() // will print 'Hi, I am a function'
```

- **Function declaration before defining.**

```
aFunction() // will print 'Hi, I am a function'
function aFunction(){
    console.log(`Hi, I am a function`);
}
```

- **Declaration after function expression**

```
const anotherFunction = function(){
    console.log(`Hi, I am another function`);
}
anotherFunction() // will print 'Hi, I am anoter function'
```

- **Declaration before function expression**

```
anotherFunction() // will give 'Uncaught ReferenceError:
Cannot access 'anotherFunction' before initialization.'

const anotherFunction = function(){
    console.log(`Hi, I am another function`);
}
```

Solve This Practice Question

You're building a **simple booking system for a movie theatre**. Create a **anonymous function** and store it in a variable **bookSeat** that takes **one parameter** called **movieName**. Inside this function, create another **named function** named as **generateTicket t**hat returns a message like **"Ticket booked for Inception"**.

- Call **generateTicket** from within **bookSeat** and return its result.
- The message in **generateTicket** should use the **movieName parameter passed to bookSeat**
- Do not call **generateTicket** outside of **bookSeat**

Made with GAMMA

Arrow Functions

- An arrow function uses **" => "** symbol.
- An arrow function is always written as a **function expression**.
- Arrow functions **allow to use shorter syntax** for function expression.
- You can **skip the function keyword** while writing the arrow function.
- **Eg :**

Normal Function Expression

```
const egFunc = function(){
   console.log('Normal function example')
}
```

Arrow Function

```
const egFunc = () =>{
   console.log('Arrow Function example')
}
```

- Arrow functions **return value by default**, if the **function has one statement that returns a value**.
- Thus in this case **you can remove curly brackets and return keyword** from your syntax.
- **Eg** :

Normal Function Expression

```
const greetFunc = function(){
    return `Greetings of the day`
}
```

Arrow Function

```
const greetFunc = () => `Greetings of the day`
```

- If you have **parameters**, you pass them **inside the parenthesis**.
- If you have **only one parameter**, you can also **skip parenthesis as well**.
- **Eg** :

Eg-1

```
const birthdayFn = (name) => `Happy birthday ${name}.`
```

Eg-2

```
const birthdayFn = name => `Happy birthday ${name}.`
```

- If you have more than one parameters, **implicit return** can be performed in the following way using **"round brackets () "**

Normal Function Expression

```
const multTwo = (numb1, numb2) => {
   return numb1*numb2
}
```

Arrow Function

```
const multTwo = (numb1, numb2) =>(numb1*numb2)
```

Solve This Practice Question

You're building a quick greeter for a hotel check-in app. Write an arrow function named **greetGuest** that takes **two parameters**, **guestName** and **roomNumber**, and returns a string like **"Welcome Alex, your room number is 101".**

- Store the arrow function in a **const** variable named **greetGuest**
- Call the function with a **name** and **room number of your choice**
- Store the **returned result** in a variable named **greeting**

Made with GAMMA

Immediately Invoked Function Expression (IIFE)

- Normally, a function runs only when it is called.
- An **IIFE runs automatically** when the **JavaScript engine reads it (compiles it)**.
- An IIFE function is **defined** and **executed** at the **same time**.
- **Eg** :

```
// named IIFE
(function hello(){
    console.log("Hello");
})();
```

```
// unnamed IIFE
(function (){
   console.log("Hello there")
})();
```

```
// unnamaed Arrow IIFE
(() => {
    console.log("Hello Now");
})()
```

```
// IIFE with parameter
((name) => {
    console.log(`Hello now ${name}`);
})('john')
```

Why is IIFE used ?

- **To create private scope** : Everything inside an IIFE is private to that function.

Eg :

```
(function(){
   let a = 25;
})()
// a is not accesible here
```

- **Avoid polluting global scope** : Variables inside an IIFE cannot be accessed from outside.

Eg :

```
(function () {
  let hidden = 42;
})();

let result = hidden; // Error: hidden is not defined.
```

- IIFE were heavily used before ES6, so working with older JavaScript code so while working with older code/ legacy code you may encounter IIFE.

Made with GAMMA

Questions for more practise

Question 1 — Restaurant Bill Calculator

You're building a billing feature for a restaurant's website. The app needs to calculate the total bill for a customer based on the items they ordered.

- Declare a regular function named **calculateBill** that takes **two parameters** — **itemPrice** and **quantity** — and a **third parameter discount** that has a **default value of 0**.
- Inside the function, **calculate the subtotal (itemPrice * quantity)**, then **subtract the discount** from it and **return the final amount**.
- Write a separate **arrow function** named **formatBill** that takes **one parameter amount** and returns a string like **"Your total bill is ₹540"**. Use the **implicit return** (shorthand).
- **Call calculateBill** with an **item price and quantity of your choice**, pass the result into **formatBill**, and store the **final formatted string** in a variable named **receipt**.
- **Call calculateBill again**, this time **without passing a discount**, to **verify the default parameter kicks in**, and **console.log that result directly**.

Question 2 — Website Startup Initialiser

You're building the initialisation logic for a startup landing page. When the JavaScript file loads, t**he page should immediately set up some config values and display a welcome banner — without polluting the global scope** with variables that don't need to be global.

- Declare a let variable named **appName** in the **global scope** and a**ssign it a name for your app (e.g. "LaunchPad")**. This represents a config value the whole app needs access to.
- **Write an IIFE (you can use either the named or arrow syntax) that runs immediately when the script loads. Inside it**, declare a let variable named **bannerMessage** that **builds a string using appName** from the global scope, something like **"Welcome to LaunchPad — We're live!"**.
- **Inside the same IIFE**, declare another **let variable named loadTime** and assign it a string like **"Loaded at 09:00 AM"**. This should stay **private to the II**FE.
- **console.log** both **bannerMessage** and **loadTime** from **within the IIFE**.
- **After the IIFE**, write a **regular named function** called **getAppName** that returns **appName (the global variable)**, and **call and console.log it below**. This shows that global scope is still reachable even after the IIFE runs.
- Finally, try to **console.log(loadTime) outside the IIFE** and **add a comment explaining what happens and why IIFE was the right choice here over a regular function**.

Question 3 — User Profile Card Generator

You're working on a social media app where each user gets a profile card when they sign up. The card generation logic needs to be properly scoped so that private details don't leak outside.

- Create an anonymous function and store it in a variable named **generateCard**. It should take **two parameters** — **username** and **role**.
- **Inside generateCard**, declare a **let variable** named **cardId** and **assign it any number** (this simulates a private internal ID **that shouldn't be accessible outside**).
- Inside **generateCard**, define a **named inner function** called **buildCar**d that **uses both username, role, and cardId (from the outer function's scope)** to return a **string** like **"Profile Card | User: alex | Role: Admin | ID: 1042"**.
- Call **buildCard** from within **generateCard** and **return its result**. **Do not call buildCard outside of generateCard.**
- **Store the result** of calling **generateCard** with a **username** and **role of your choice** in a variable named **myCard** and **console.log** it.
- After all this, **try to console.log(cardId) outside the function and add a comment explaining what error you'd expect and why**.

Made with GAMMA

Objects in JS

Lec-7

Made with GAMMA

Objects in JS

- Objects are **variable** that can store both **values** and **functions**.
- Values are stored as **key: value** pairs called as **properties**.
- Objects by default stores key and value in Strings.
- Functions are stored as **key:function()** pairs called **methods**.



Phone Properties	Phone Methods
phone.name = iPhone	phone.on()
phone.model = 17 Pro	phone.play()
phone.weight = 206 gm	phone.pause()
phone.color = Cosmic Orange	phone.off()

phone Object

- Different phones have the same **properties**, but the **property values** can differ from phone to phone.
- Different phones can have the same **methods**, but the methods can be performed **at different times**.
- **Eg** :

```
const phone = {
    name: "iPhone",
    model: "17 Pro",
    color: "Cosmic Orange"
};
```

- Here, **name**, **model** and **color** are **properties** and **"iPhone"**, **"17 Pro"**, **"Cosmic Orange"** are **property values**.
- Generally Objects can be created by the following 2 ways:

Way-1 : Using Object literal.

Way-2 : Using 'new' keyword

Using Object literal

```
const person = {
    firstName: "John",
    lastName:  "Doe",
    age: 18,
    sex: "male",
}
```

Using 'new' keyword

```
const person = new Object({
    firtName: "John",
    lastName: "Doe",
    age: 18,
    sex: "male"
});
```

Adding Properties in an object

- You can also create an **empty object**, and add properties later.
- **Eg**:

```
const person = {}
console.log(person);
person.firstName = "John"
person.lastName = "Doe"
person.age = 18
person.sex = "male"
```

Deleting Properties of an object

- The delete keyword deletes a property from an object
- **Eg**:

```
delete person.sex;
console.log(person);
```

Check if a Property Exists in an object

- Use the **in** property to check if a property exists in an object or not.
- **Eg**:

```
console.log("age" in person); // will print true
```

Made with GAMMA

Accessing Object Properties

- You can access object properties in two ways:

1. **Dot notation**
2. **Bracket notation**

- **Eg**:

```
const car = {
   type: "Ferrari",
   model: "420 GTB",
   "car color": "Red",
   weight: "1,720kg"
}
```

```
console.log(car.type); // Ferrari
console.log(car.weight); // 1,720kg
```

Here accessing car color is not possible with the help of dot notation

```
console.log(car.car color); // This will result in SyntaxError
```

In such cases we tend to use bracket notation

```
console.log(car["car color"]); // Red
```

Preventing Changes in our Object Properties

- In order to prevent any changes to take place in our existing object, we use **Object.freeze()** method.
- **Eg** :

```
const movieObj = {
   "movie name": "Interstellar",
   "IMDB ratings": 8.7,
   "Released Year": 2014,
   "Streaming Platform": "Amazon Prime"
}
```

```
console.log(movieObj["Streaming Platform"]);
// will print Amazon Prime

movieObj["Streaming Platform"] = "Netflix"
console.log(movieObj["Streaming Platform"]);
// will print Netflix
```

Now let's freeze the object

```
Object.freeze(movieObj)

movieObj["Released Year"] = 2016
console.log(movieObj["Released Year"]);
// will result in typeError
```

> **Note** :
>
> - If an object is freezed and if we try to change any of it's properties, doing so will result in **typeError** only when **"use strict"** is used at the top of your script file.
> - Incase **"use strict"** is not used, it would **not result in typeError** instead it would **print the old value itself**.

- In order to know any object is in freezed state or not, we can use **Object.isFrozen()**, which returns boolean value i.e either **true** or **false**.
- **Eg** :

```
console.log(Object.isFrozen(movieObj));
// will return true
```

- Suppose you want to get all the **keys** of an object
- **Eg** :

```
console.log(Object.keys(movieObj));
// ['movie name', 'IMDB ratings', 'Released Year', 'Streaming Platform', 'watched']
```

- Suppose you want to get all the **values** of an object
- **Eg** :

```
console.log(Object.values(movieObj));
// ['Interstellar', 8.7, 2014, 'Netflix', ƒ]
```

- Suppose you want to get all the **entries** of an object.
- **Eg** :

```
console.log(Object.entries(movieObj));
// ['movie name', 'Interstellar']
// ['IMDB ratings', 8.7]
// ['Released Year', 2014]
// ['Streaming Platform', 'Netflix']
// ['watched', ƒ]
```

> **Note** : **Object.keys(), Object.values() and Object.entries() method return and store thier result in an Array.**

Made with GAMMA

'this' keyword in JS

- The **this** keyword **refers to an object**.
- In Js, **this** is used to access the object that is calling a method.
- Whenever **this** is used inside an object method, **this** refers to the object.
- **Eg** :

```
const personDet = {
    firstName: "John",
    lastName: "Doe",
    age: 25,
    hellofunc: function(){
        console.log(`Weclome ${this.firstName} ${this.lastName}`);
    }
}
console.log(personDet.hellofunc());
```

- Here **firstName** and **lastName** cannot be accessed inside our **hellofunc()** without using this keyword since they're the properties of an object personDet and can only be accessed with the help of this keyword.
- **this.firstName** refers to the **firstName** property of the **personDet** object.
- **this.lastName** refers to the **lastName** property of the **personDet** object.
- Another reason for using **this** keyword, is we can use same method with different objects.
- **Eg** :

```
const person1 = {
    firstName: "Joe",
    lastName: "Biden",
    age: 69,
    hellofunc: function(){
        console.log(`Weclome ${this.firstName} ${this.lastName}`);
    }
}
console.log(person1.hellofunc());
// will print Welcome Joe Biden
```

```
onst person2 = {
    firstName: "Donald",
    lastName: "Duck",
    age: 65,
    hellofunc: function(){
        console.log(`Weclome ${this.firstName} ${this.lastName}`);
    }
}
console.log(person1.hellofunc());
// will print Welcome Donald Duck
```

- Whenever we use **this** keyword alone, it refers to the **global / supermost object**.
- In a browser, the supermost object is **window** object.
- In **node environment**, when used **this** keyword **alone** it refers to a **blank object {}**.
- If we use **arrow functions** inside an object, then it would give unexpected result compared to the above example.

```
const personDet = {
    firstName: "John",
    lastName: "Doe",
    age: 25,
    hellofunc: () =>{
        console.log(`Weclome ${this.firstName} ${this.lastName}`);
    }
}
console.log(personDet.hellofunc());
// will print "Welcome undefined undefined"
```

- In above example, **this** keyword does not refer to the personDet object.
- This is because arrow functions do not have thier own **this** value.
- Thus, **it's better to avoid using arrow functions inside an object**.

Made with GAMMA

Practice Questions

Question-1 : E-commerce Inventory Tracker

You're adding a small inventory feature to a store project. Each product needs to live as an object so the catalog page can display its details and the warehouse team can update stock without touching the code directly.

- Create a product object literal named product with properties: **name, brand, price, stock count**, and **category**.
- Log the product's **name** to the console.
- A new shipment just arrived — add a new property called **restockDate** to the object.
- The product just got discontinued, so delete the **category** property from the object. Before deleting it, check whether a **warranty** property exists on the object and log the result.
- Finally, log the product's full structure **(keys, values and entries)**

Question 2: Player Bio Cards for Your World Cup Predictor

You're extending your World Cup Predictor project to show a short bio whenever a player card is clicked. Instead of writing a separate console log for every player, you want one reusable method that adapts based on whichever object calls it.

- Create two player objects, e.g. **player1** and **player2**, each with **name, club, position**, and **goals** properties.
- Give **each object** a **method** called **bio** that uses this to **log** something like **"Vinícius Júnior plays as a Forward for Real Madrid and has scored 24 goals this season."**
- Call bio() on both objects and confirm the same method logic produces different output depending on which object called it.

Question 3: Locking In a Flight Booking Price

You're building a flight booking feature where, once a customer confirms their seat, the displayed price should never silently change due to a bug somewhere else in the app, even if some other function tries to touch it.

- Create a booking object with **passengerName**, **flightNumber**, **seat**, and **price** properties.
- Log the **price** to the console
- Inside a variable **isBookingConfirmed** store the result on basis of booking is confirmed or not
- Freeze the object with once the booking is confirmed, and verify it's frozen or not.
- Add **"use strict"** at the top of your file, try changing the price one more time, and describe what error shows up and why the behavior changed.

Made with GAMMA

Loops in JS

Lec-8

Made with GAMMA

Loops in JS

- Loops can execute a block of code a number of times.
- Loops are handy, if you want to run the same code over and over again, each time with a different value
- JavaScript has following types of loops:

1. while loop
2. do while loop
3. for loop

Made with GAMMA

While loop

- While loops execute a block of code **as long as a specified condition is true**.
- **Syntax** :

```
while(condition){
// code to be executed
}
```

- Suppose we're printing numbers from 1 to 10 using while loop

```
let num = 1
while (num <= 10) {
    console.log(num);
    num+=1
}
// Output: 1 2 3 4 5 6 7 8 9 10
```

- Suppose we're iterating over a array using while loop

```
let arr1 = ['Apple','Oranges','Banana','Jackfruit']
let itevar = 0
while (itevar < arr1.length){
    console.log(arr1[itevar]);
    itevar+=1
}
// Output: Apple Oranges Banana Jackfruit
```

Note :

- If you forget to increase the variable used in the condition, the loop will never end.
- This will crash your browser.

Practice Question-1

A football club's new signing has been given a fitness target by the coach. He needs to complete several training sessions before he's cleared to play in the next match, and the club wants a simple program to track his progress session by session.

Create a variable to track the number of sessions completed, **starting at 0** Use a **while loop** that **keeps running until he reaches 5 completed sessions** On each loop iteration, **increase the session count by 1** and print a message showing which session number he just finished Once the loop ends, print a final message saying **"player is fit and ready to play."**

The Output should be in a following way:

Player has completed 1 / 5 training sessions.

Player has completed 2 / 5 training sessions.

Player has completed 3 / 5 training sessions.

Player has completed 4 / 5 training sessions.

Player has completed 5 / 5 training sessions.

The Player is fit and ready to play

Practice Question-2

You're building a quick squad-check tool for your FIFA World Cup 2026 Predictor project. You have a list of player names for one of the teams, and you want to search through it using a while loop to find out if a specific player is part of the squad — **without using any array search methods like includes() or indexOf()**.

Create an array of at least 6 player names - **['m. salah','vini jr','haaland','mbappe','neymar','messi','ronaldo']**

Create a variable to hold the name of the player you're searching for

Use a while loop with an index variable to go through the array one player at a time Inside the loop, check if the current player matches the one you're searching for — if it matches, print that the player was found and stop checking further players

If the loop finishes without finding a match, print a message saying the **Player not available.**

Made with GAMMA

Do While loop

- The **do while** loop is a variant of the while loop.
- This loop will **execute the code block once**, **before checking if the condition is true**, then it will **repeat the loop as long as the condition is true**.
- The **do while loop at least runs at once**, even if the condition is **false from the start**.
- This is because the **code block is executed before the condition is tested**.
- Do not forget to increase the variable used in the condition, otherwise the loop will never end!
- **Syntax** :

```
do {
  // code block to be executed
}
while (condition);
```

- Suppose we're printing numbers from 1 to 10 using do while loop

```
let variab = 1
do {
    console.log(variab);
    variab+=1
} while (variab <= 10);
```

- Suppose If we put value of variab as 11 instead of 1 in same code

```
let variab = 11
do {
    console.log(variab);
    variab+=1
} while (variab <= 10);
```

Above code will print 11 eventough the value stored in variab contradicts the given condition, this is because do while loop runs atleast once before checking the condition.

Practice Question

A local coffee shop gives out a stamp for every coffee a customer buys, and once they collect enough stamps, they get a free coffee. You're building a quick simulation to see how many coffees a customer buys before earning their free one.

- Create a variable to track the number of stamps collected, starting at 0
- Use a **do while loop** so the customer always buys at least one coffee before any check happens
- Inside the loop, increase the stamp count by 1 and print a message showing the current stamp count after each purchase
- Keep the loop running as long as the stamp count is less than 6
- Once the loop ends, print a message saying the customer has earned a free coffee, along with the total number of coffees it took

Made with GAMMA

For Loop

- The **for** statement creates a loop woth 3 optional expressions:
- **Syntax** :

```
for (initiation; condition; incrementation/decrementation)
{
   // code block
}
// initiation: code starts executing from here

// condition : condition to be tested before every execution

// increment/decrement : executes every time after each code block is done executing and changes the value before the next execution.
```

Practice Question - 1

You're managing the playlist for a friend's birthday party, and you've got a list of songs queued up. Before the party starts, you want a quick script that announces each song in order as it's about to play, along with its position in the queue.

Create an **array of at least 5 song names (or song + artist pairs, your choice)** Use a **for loop** to go through the array from start to finish On each iteration, print a message showing the song's position in the queue (like **"Song 1: ...") along with the song name** After the loop finishes, print a final message saying the **playlist has ended, along with the total number of songs that played**

Practice Question - 2

You're setting up seating for a wedding reception. There are several tables, and each table has a fixed number of seats. You want a script that prints out a seating label for every single seat at every table, so the venue staff can stick labels on each chair.

Create a variable for the number of tables (for example, 4), Create a variable for the number of seats per table (for example, 6). Use an **outer for loop to go through each table Inside it**, **use an inner for loop to go through each seat at that table** On each inner iteration, print a label like **"Table 1 - Seat 1", "Table 1 - Seat 2"**, and so on After each table's seats are fully labeled, **print a short separator message** before moving to the next table Once both loops finish, print a final message showing the total number of seats labeled across the whole venue

Made with GAMMA

Break and Continue

- The **break** statement **"jumps out"** of the loops and switches.
- When **break** is encountered in a loop, **the loop terminates immediately**.
- The program **control is transfered to the statements following the loop**.
- **No more loop iterations are executed**.

Practice Question-1

You're checking attendance for a college lecture. **You have a list of registered student names, and the moment you spot a particular student you're looking for, you don't need to keep checking the rest of the list — you can just stop right there and move on.**

Create an **array of at least 6 student names** Create a **variable holding the name of the student you're looking for (make sure it's somewhere in the array)** Use a for loop to go through the array from start to finish **On each iteration, print the name of the student currently being checked As soon as you find a match with the student you're looking for**, print a message saying they've been found, along with their position in the list **Use the break keyword right after finding the match, so the loop stops checking the remaining students immediately** After the loop, **print a message confirming attendance check is complete**

- The **continue** statement **skips the current iteration** in a loop.

Practice Question-2

You're going through a **list of grocery items on your shopping app to print a packing slip**, but a **few items in the list are out of stock and marked as null**. You don't want those showing up on the packing slip at all — **you just want to skip over them and continue printing the rest**.

Create an array of at least 7 items where a couple of the entries are null (representing out-of-stock items) and the **rest are item names as strings** Use a **for loop to go through the array from start to finish Inside the loop, check if the current item is null — if it is, use the continue keyword to skip straight to the next iteration without printing anything** If the **item isn't null, print a message showing its position in the list along with the item name**, as if it's being added to the packing slip **After the loop finishes, print a final message saying the packing slip is ready, along with the total number of valid items that were printed (not counting the skipped ones)**

- **break** and **continue** are the only JavaScript statements that can **"jump out of"** a code block.

Made with GAMMA

for ...of Loop

- The **for ...of** loop iterates over the values of an iterable objects such as arrays, strings, etc.
- **Syntax** :

```
for (iterating_variable of iterable){
  // code block to be executed
}
```

- Suppose you have an array consisting of some numbers you can print those numbers one by one using for ...of loop

```
const numArr = [56, 5, 96, 102, 89]

for (const i of numArr) {
console.log(i);
}
```

Practice Question

You're building a quick feature for your FIFA World Cup 2026 Predictor app that announces the qualified teams as they get confirmed, one by one, for a highlight reel shown on screen.

- Create an **array of at least 6 country names** that have qualified for the tournament
- Use a **for...of loop to go through the array**
- On each iteration, **print a message announcing that team as "qualified" (for example, "Brazil has qualified!")**
- Keep a counter variable outside the loop that increases by 1 on every iteration, so you can track how many teams have been announced so far
- After the loop finishes, print a final message showing the total number of qualified teams announced

Made with GAMMA

for ...in Loop

- The **for ...in** loop iterates over the enumerable properties of an object.
- It is typically used for iterating over object keys.
- **Syntax** :

```
for (key in object){
  // code block to be executed
}
```

- Suppose you are iterating over a following object

```
const langObj = {
   js: 'javaScript',
   cpp: 'C ++',
   java:'java',
   rb:'ruby'
}
```

- This can be done in a following way using **for ...in** Loop

```
for(const key in langObj){
   console.log(`${key} shortcut is ${langObj[key]}`);
}
```

Practice Question

You're building a quick feature for your FIFA World Cup 2026 Predictor app You're building a quick player profile card for your FIFA World Cup 2026 Predictor app, and you want a script that prints out every piece of information stored about a player, without having to write out each property by name.

- Create an object representing a player with at least 5 properties (for example: name, country, position, age, goals)
- Use a for...in loop to go through the object
- On each iteration, print the property name along with its value, like "name: Vinicius Jr"
- Keep a counter variable outside the loop that increases by 1 on every iteration, so you can track how many properties were printed in total
- After the loop finishes, print a final message showing the total number of properties that were displayed on the profile card

Made with GAMMA

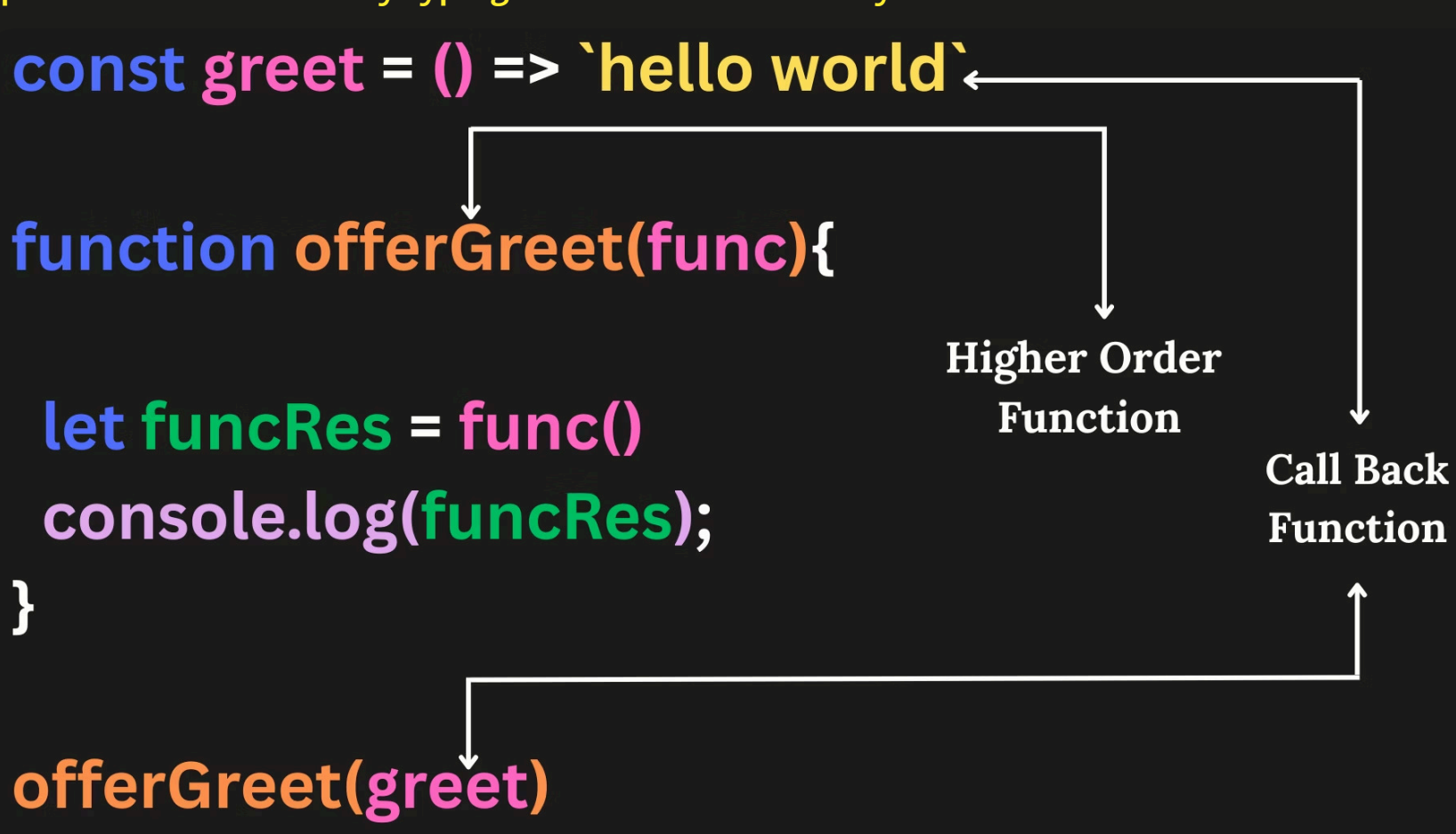
Iterators in JS

Lec-9

Made with GAMMA

Higher Order Functions

- In JavaScript, **functions are first class objects**, this means that like other objects you've encountered JavaScript functions can have properties and methods.
- Functions are special because we can invoke them, but **we can still treat them like any other type of data**.
- A Higher Order Function is a function that **either accepts functions as a parameter**, **returns a functions** or **does both**.
- The functions which get passed in as parameters are called as - **callback functions**, they **get invoked** during the execution of higher-order function.
- When we **invoke a higher-order function**, and **pass another function as an argument to the callback function** while doing so **we don't invoke the argument to the callback function using open close paranthesis '()'**, as invoking it would also evaluate to pass return value of that function call. Thus we have to **pass in function itself by typing the function name only**.



- Eg :

```
const birthYear = (byear) => byear

function currentAge(func, yearVal, cYear = 2026){
    return `Your current age is : ${cYear - func(yearVal)}`
}
let ageRes = currentAge(birthYear, 2001);
console.log(ageRes);
```

Practice Question

Create a function **annualSal** which takes **asalary** as a parameter and returns it.

Create another function **monthlyCredit** which takes 3 parameters- **salaryFunc, asal, pf** which returns the monthly salary crediting into a person's account.

Inside **monthlyCredit()** pass **asal** as a parameter to the **salaryFunc** and divide it by 12 also subtract 1200 from it as a part of PF contribution.

Display the result by calling **monthlyCredit()** with accurate arguments matching those parameters.

Made with GAMMA

Introduction to Iterators

- Iterators are **built in JavaScript array methods**, that helps us iterate over an array.
- Iterators are **methods** called on **arrays to manipulate eleements are return values**.
- All Iterator methods take's **callback function** as an **argument** to **one of their parameters**.

1. .forEach()

- **.forEach()** will execute same code for each element of an array.
- A **callback function** is passed as an **argument** inside **.forEach()**
- **.forEach()** loops through the array and **executes the callback function for each element of that array**.
- During **each execution**, the **current element is passed as an argument to the callback function**.
- The **return value** of **.forEach()** method is always **undefined**.
- **Eg** : Suppose you have an array of some numbers and you wish to print the halves of those numbers consisting in that array, this can be achieved in a following way:

```
const randomNumbers = [80, 40, 20, 10]
randomNumbers.forEach(function iterFunc(numIter){
    console.log(numIter);
})

// This will print 80 40 20 10 to the console.
```

- We can also print indexes and complete array using **.forEach()**

```
const randomNumbers = [80, 40, 20, 10]

randomNumbers.forEach(function iterFunc(numIter,index){
    console.log(`${numIter} is at ${index} index`);
})
/* This will print
   80 is at 0 index
   40 is at 1 index
   20 is at 2 index
   10 is at 3 index */

const randomNumbers = [80, 40, 20, 10]

randomNumbers.forEach(function iterFunc(numIter,index,arr){
    console.log(arr);
})
/* This will print
   [80, 40, 20, 10]
   [80, 40, 20, 10]
   [80, 40, 20, 10]
   [80, 40, 20, 10] */
```

- Suppose we have an array consisting of multiple objects inside it, so by using **.forEach()** we can access each and every property of an array according to our need.

```
const mobDetails = [
    {
        brand: "Apple",
        model: "Iphone",
    },
    {
        brand: "Samsung",
        model: "S25",
    },
    {
        brand: "One Plus",
        model: "Nord CE5",
    }
]
mobDetails.forEach(function(mobItem){
    console.log(mobItem.brand);
    /* This will print :
       Apple
       Samsung
       One Plus */
})
```

Practice Question

You're building a simple movie watchlist feature. You have an array of movies a user has saved to watch later, and you want to display them in the console in a readable format — no filtering, no transforming, just looping through and doing something with each item. Here's your data:

c**onst watchlist = [{ title: "Inception", genre: "Sci-Fi", watched: false }, { title: "The Dark Knight", genre: "Action", watched: true }, { title: "Interstellar", genre: "Sci-Fi", watched: false }, { title: "Parasite", genre: "Thriller", watched: true }, { title: "Dune", genre: "Sci-Fi", watched: false }];**

Use **.forEach()** to loop through the watchlist and log each movie's title and genre in this format: **"Inception - Sci-Fi"** Modify your .forEach() to also print a status alongside each movie — if watched is true, add "✅ Watched", otherwise add "⏳ Pending". Example output: **"Inception - Sci-Fi - ⏳ Pending"** Bonus: Inside the same .forEach(), **keep a count of how many movies are still pending (not watched yet) and log the total count after the loop finishes**

Made with GAMMA

Introduction to Iterators

2. .filter()

- **.filter()** method takes argument as a **callback function**, just like **.forEach()**.
- Similarly like **.forEach()** method **.filter()** also loops through a particular array and returns it's elements.
- The only difference is unlike **.forEach()** method, **.filter()** method **returns the element in new array**.
- Suppose, we have an array and we have to print only those element of an array which are less than 10, by filtering them
- **Eg** :

```
const numArr = [36, 41, 2, 99, 6, 4, 66, 1]
let numArrFilter = numArr.filter((num) => num < 10)
console.log(numArrFilter); // will print [2,6,4,1]
```

```
const mobileApp = [
    {
        appName: 'Whatsapp', launched: 2009, "recent version": "2.26.24.81",
    },
    {
        appName: 'Snapchat', launched: 2011, "recent version": "14.12.0.46",
    },
    {
        appName: 'Instagram', launched: 2010, "recent version": "436.0.0.41",
    },
    {
        appName: 'Facebook', launched: 2004, "recent version": "576.0.0",
    },
    {
        appName: 'Telegram', launched: 2013, "recent version": "12.8.3",
    }
]

let filteringOnLaunch = mobileApp.filter((data) => data.launched < 2010)
console.log(filteringOnLaunch);
/* This will print
{appName: 'Whatsapp', launched: 2009, recent version: '2.26.24.81'}
{appName: 'Facebook', launched: 2004, recent version: '576.0.0'}
*/
```

Practice Question

You're working on a basic task manager for a dev team. The team has a list of tasks logged in the system, and you need to pull out specific subsets of tasks depending on what someone wants to view — that's exactly what .filter() is for. Here's your data:

const tasks = [{ id: 1, title: "Fix login bug", status: "done", priority: "high" }, { id: 2, title: "Write unit tests", status: "pending", priority: "medium" }, { id: 3, title: "Update README", status: "pending", priority: "low" }, { id: 4, title: "Deploy to staging", status: "done", priority: "high" }, { id: 5, title: "Code review PR #12", status: "pending", priority: "high" }, { id: 6, title: "Fix CSS on mobile", status: "done", priority: "medium" }];

Use **.filter() to get all tasks where status is "pending" and log the result**. Use **.filter() to get all tasks where priority is "high" and log the result**. **Chain both conditions together in a single .filter() call — tasks that are both "pending" AND "high" priority.** These would be the most urgent ones to tackle.

Made with GAMMA

Introduction to Iterators

3. .map()

- **.map()** method takes argument as a **callback function**, just like **.forEach()** and **.filter()**
- .**map()** works in a **similar manner as of .forEach()**, but the only difference is that **.map() returns it's result in a new array**.
- Also **.forEach()** method **doesn't return anything** it's **return value is undefined**, whereas **.map()** method **returns a new array**.
- **Eg**:

```
const numberss = [1, 2, 3, 4, 5, 6, 7, 8, 9]

let numPlusTen = numberss.map((num) => num + 10)

console.log(numberss); // [1, 2, 3, 4, 5, 6, 7, 8, 9]

console.log(numPlusTen); // [11, 12, 13, 14, 15, 16, 17, 18, 19]
```

- Chaining through **.map()** and **.filter()**

```
const fewNums = [1, 2, 3, 4, 5]

let fewNumsChaining = fewNums
                    .map((num) => num * 10)
                    .map((num) => num + 5)
                    .filter((num) => num % 3 == 0)

console.log(fewNumsChaining); // [15,45]
```

Practice Question

You're building a small feature for a developer portfolio site. You have a list of projects stored as raw data objects, and you need to transform them into different shapes depending on what the UI needs to display — that's exactly the job of .map(). Here's your data:

const projects = [{ title: "weather app", tech: "JavaScript", stars: 12 }, { title: "portfolio site", tech: "HTML/CSS", stars: 34 }, { title: "chat bot", tech: "Python", stars: 8 }, { title: "expense tracker", tech: "React", stars: 21 }, { title: "url shortener", tech: "Node.js", stars: 5 }];

Use .map() to return a new array of just the project titles, but with every title converted to Title Case (e.g. "weather app" → "Weather App"). Log the result.

Use .map() to return a new array of strings in this format: "weather app — Built with JavaScript". Log the result.

Use .map() to return a new array of objects where each object has just two properties — title (in Title Case) and a new property popular which is true if the project has more than 10 stars, and false otherwise.

Made with GAMMA

Introduction to Iterators

4. .reduce()

- The **.reduce()** method **returns a single value after iterating** through the elements of an array, **thereby reducing the array**.
- **Syntax** :

```
array.reduce((accumulator, currentValue) => {
  return accumulator + currentValue;
});
```

or

```
array.reduce((accumulator, currentValue) => {
  return accumulator + currentValue;
}, initialValue);
```

- **Eg** :

```
const randNums = [5,7,2,1]

let randomSum = randNums.reduce((accumulator,currentValue) =>{
    return accumulator + currentValue
})
console.log(randomSum); // 15
```

Let's see working of the above code

Iteration	accumulator	currentVal	return
first	5	7	12
second	12	2	14
third	14	1	15

- The **.reduce()** also takes an **optional second parameter** to set an **inital value for accumulator** (first element is callback function)

```
let diffRandomSum = randNums.reduce((accumulator, currentValue) =>{
    return accumulator + currentValue
},100)
console.log(diffRandomSum); // 115
```

Let's see working of the above code

Iteration	accumulator	currentVal	return
first	100	5	105
second	105	7	112
third	112	2	114
fourth	114	1	115

```
const shoppingCart = [
    {
        item: "T-Shirts",
        price: 899
    },
    {
        item: "Watch",
        price: 599
    },
    {
        item: "Power bank",
        price: 2200
    }
]

let checkOut = shoppingCart.reduce((accumulator,shoppingItems) => accumulator + shoppingItems.price,0)
console.log(checkOut);
```

Practice Question

You're building the checkout page for a small e-commerce app. You have a list of items in the user's cart, and you need to calculate a few summary numbers to display in the order summary — tallying up values from a whole list into a single result is exactly what .reduce() is built for. Here's your data:

const cart = [{ name: "Mechanical Keyboard", price: 80, quantity: 1 }, { name: "USB Hub", price: 25, quantity: 2 }, { name: "Mousepad", price: 15, quantity: 3 }, { name: "Webcam", price: 60, quantity: 1 }, { name: "LED Desk Lamp", price: 20, quantity: 2 }];

Use .reduce() to calculate the total price of the entire cart. Keep in mind each item has a price and a quantity, so the cost per item is price × quantity.

Log the final total. Use .reduce() to calculate the total number of items in the cart (sum of all quantities).

Log the result. Use .reduce() to find the most expensive item (by unit price) in the cart and log just that item's object.

Made with GAMMA

Introduction to Iterators

5. .findIndex()

- **.findIndex()** returns the **index of the first element** that **evaluates to be true in callback function**.
- If there **isn't a single element in the array that satisfies the condition** in the callback, then **.findIndex()** will return **-1**.
- **Eg**:

```
const jumbledNums = [420, 365, 961, 85, 123, 66]

let jumbledOp = jumbledNums.findIndex((num) => num < 100)
console.log(jumbledOp); // 3
```

If no element satisfies the condition:

```
const jumbledNums = [420, 365, 961, 850, 123, 660]

let jumbledOp = jumbledNums.findIndex((num) => num < 100)
console.log(jumbledOp); // -1
```

6. .some()

- **.some()** checks **if atleast one element in an array satisfies a given condition**.
- **.some()** method **takes callback function as it's main parameter**.
- The **.some()** returns a **boolean value**:

1. **true** : if **callback function returns true for atleast one element** in the array.
2. **false** : if **callback function returns false for all elements**.

- The .some() method is particularly useful **when you need to check if any element in an array meets a certain condition**, **without having to iterate through the entire array manuallly**, It **stops execution as soon as it finds an element that satisfiess the condition**, making it efficient for large arrays.

```
const jumbledNums = [420, 365, 961, 85, 123, 66]
let checkNum = jumbledNums.some((num) => num % 2 == 0)
console.log(checkNum);
```

Practice Question

You're building a simple user management feature for a web app's admin panel. The admin needs to quickly check things about the registered users — like whether a certain type of user exists, or where a specific user sits in the list — without looping through everything manually. Here's your data:

const users = [{ id: 101, name: "Alice", role: "admin", active: true }, { id: 102, name: "Bob", role: "editor", active: false }, { id: 103, name: "Charlie", role: "viewer", active: true }, { id: 104, name: "Diana", role: "editor", active: true }, { id: 105, name: "Evan", role: "viewer", active: false }];

Use .some() to check if at least one user is currently inactive (active: false). Log true or false.

Use .some() to check if there is any admin in the list. Log true or false.

Use .findIndex() to find the index position of the user named "Diana" in the array. Log the index.

Use .findIndex() to find the index of the first inactive user. Log the index.

Use .findIndex() to search for a user with id: 999 — someone who doesn't exist — and log a message like "User not found" if the result is -1.

Made with GAMMA

Introduction to DOM

Lec-10

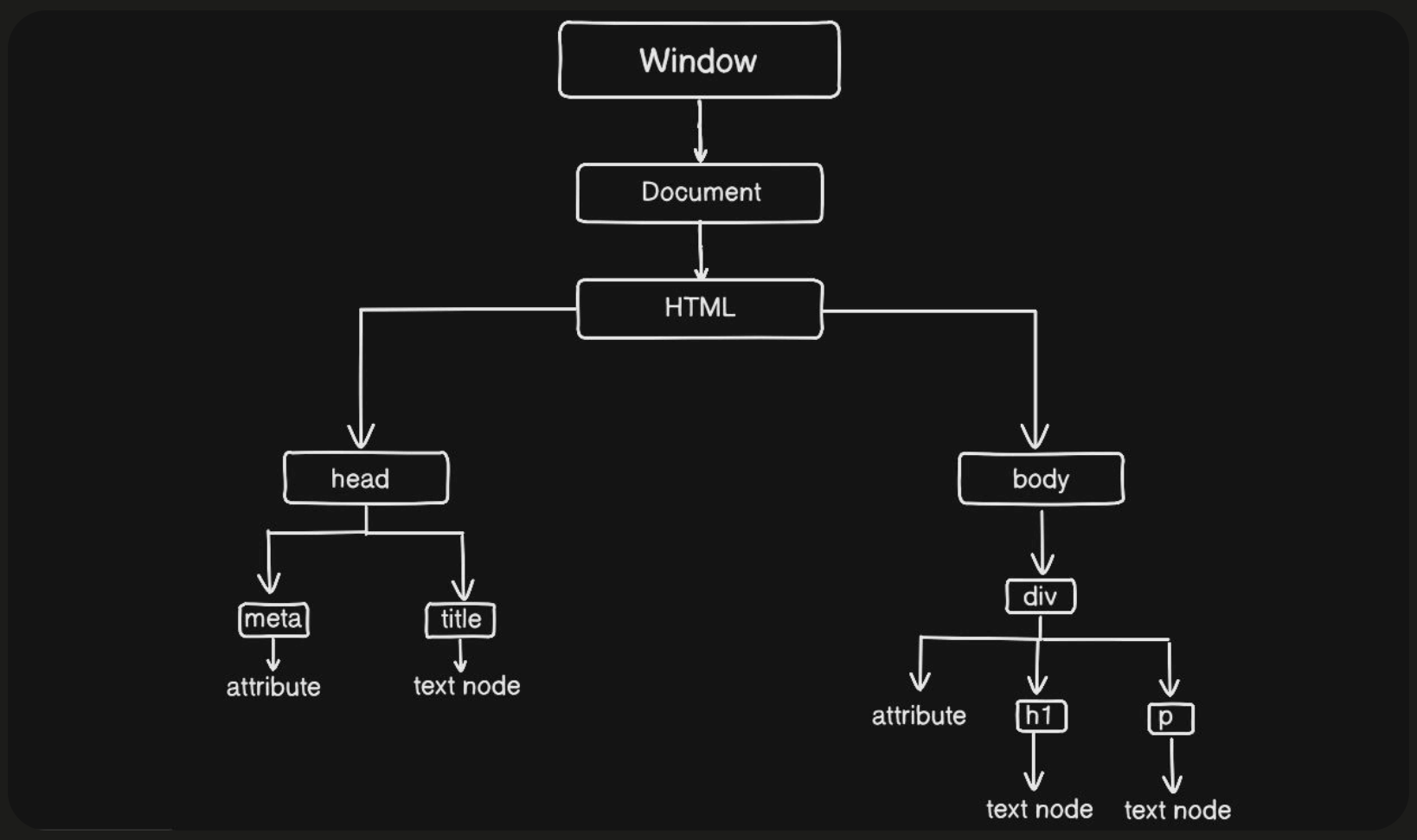
Made with GAMMA

Document Object Model (DOM)

- The HTML Document Object Model **(DOM) is an object model for HTML Documents**.
- HTML DOM is a **tree of nodes that represent an HTML page**.
- Suppose we have a following HTML code.

```
<!doctype html>
<html lang="en">
 <head>
  <meta charset="UTF-8" />
  <title>DOM</title>
 </head>
 <body>
  <div class="bg-black">
   <h1>Intro To DOM</h1>
   <p>Lorem ipsum dolor sit amet.</p>
  </div>
 </body>
</html>
```

above code can be represented in a DOM Tree Structure by following way



- The DOM API provides us with the ability to:

1. **Find and select elements.**
2. **Change element content and attributes.**
3. **Add, remove, or modify elements.**
4. **Change CSS styles.**
5. **Add event listeners to react to user input.**

- Is a standard for how to get, change, add, or delete HTML DOM element.
- Using JavaScript we can access the DOM through the API inside our browser.

Made with GAMMA

DOM API Methods and properties

- Developers use global objects like **document** and **window** as entry points to any API.
- If you want to access any element in an HTML page, you always start with accessing the document object. The **document object** represents your **web page.**
- To manipulate HTML with JavaScript, you first need to **select an element**.

Selecting DOM elements

1. getElementById()

- The easiest way to find an HTML element in the DOM, is by **using an element id**.
- This eg find the element with **id="first-head"**.
- **Eg**:

HTML

```
<h1 id="first-head">
Lorem ipsum dolor sit amet.
</h1>
```

JS

```
console.log(document.getElementById('first-head'))

/* Output:
<h1 id="first-head">Lorem ipsum dolor sit amet.</h1>
*/
```

- If the **element is found**, the **method will return the element as an object** (in element).
- If the **element is not found**, element will contain **null.**

2. getElementByTagName()

- Returns an **HTML collection** of **all the elements using that specific HTML tag** .
- This eg find the **<p>** elements.
- In order to **iterate over the HTML collection** it **first needs to be converted into an array**.
- **Eg**:

HTML

```
<p>Lorem ipsum dolor sit amet.</p>

<p id="main-para">Lorem ipsum dolor sit, amet
consectetur adipisicing elit. Ipsa, ab?.</p>

<p>Lorem ipsum dolor sit amet.</p>

<p>Lorem ipsum dolor sit amet.</p>
```

JS

```
console.log(document.getElementsByTagName('p'))
/*
Output:
HTMLCollection(4) [p, p#main-para, p, p, main-para:
p#main-para]
*/
```

- Below eg finds all **<p>** tags inside a ** ** with an **id="container-1"**.
- **Eg**:

HTML

```
<div id="container-1">

  <h1>Lorem, ipsum dolor.</h1>
  <p>Lorem ipsum dolor sit amet.</p>
  <h1>Lorem ipsum, dolor sit amet consectetur
adipisicing elit.</h1>
  <p>Lorem ipsum dolor sit amet, consectetur
adipisicing elit. Itaque, quos.</p>

</div>
```

JS

```
let containerVar1 =
document.getElementById('container-1')

console.log(containerVar1.getElementsByTagName('p'))

/*
Output:
HTMLCollection(2) [p, p]
*/
```

Made with GAMMA

Selecting DOM Elements

3. getElementByClassName()

- It is used to find all HTML elements with the **same class name**.
- It also **returns the result in the form of HTML collection**.
- In order to **iterate over the HTML collection** it **first needs to be converted into an array**.

HTML

```
<ul>
  <li class="intro">Intro One</li>
  <li class="intro">Intro Two</li>
  <li class="intro">Intro Three</li>
</ul>
```

JS

```
console.log(document.getElementsByClassName('intro')
)

/* Output:
HTMLCollection(3) [li.intro, li.intro, li.intro]
*/
```

4. querySelector()

- Returns the **first element** that matches a specified CSS selector string.
- This works easily with **Element selectors**, **Class selectors**, **Id selectors** and **attribute selectors** of CSS.

HTML

```
<form action="submit">

  <label for="name">Name</label>
  <input type="text" name="" id="" class="input">
   <br>
   <br>
  <label for="email">Email</label>
  <input type="email" name="" id="" class="input">
   <br>
   <br>
  <button class="btn">Submit</button>

</form>
```

JS

```
console.log(document.querySelector('.input'))

/*Output:
<input type="text" name="" id="" class="input">
*/
```

5. querySelectorAll()

- Returns a **static NodeList** containing **all elements** matching the given CSS selector.
- This works easily with **Element selectors**, **Class selectors**, **Id selectors** and **attribute selectors** of CSS.
- It supports forEach() loop without converting the nodeList into an array first

HTML

```
<form action="submit">

  <label for="name">Name</label>
  <input type="text" name="" id="" class="input">
   <br>
   <br>
  <label for="email">Email</label>
  <input type="email" name="" id="" class="input">
   <br>
   <br>
  <button class="btn">Submit</button>

</form>
```

JS

```
console.log(document.querySelectorAll('.input'))

/* Output:
NodeList(2) [input.input, input.input]
*/
```

Made with GAMMA

Properties to Change HTML contents

1. document.innerHTML

- Returns the entire HTML markup found inside the targeted element.
- It includes all HTML tags, attributes, and text. Writing to this property compiles and renders new HTML elements.

HTML

```
<h1 id="demo">Lorem ipsum dolor sit amet consectetur adipisicing elit.
<a href="#" target="_blank" style="display: none;">Link</a> ipsum maiores inventore eveniet!</h1>
```

JS

```
let headingOne = document.getElementById('demo')

console.log(headingOne.innerHTML)

/*
 Output:

'Lorem ipsum dolor sit amet consectetur adipisicing elit. <a href="#" target="_blank" style="display: none;">Link</a> ipsum maiores inventore eveniet!'

*/
```

2. document.innerText

- Returns only the **visible text content that appears on the browser screen**.
- It respects CSS styling and hides elements with **display: none.**

HTML

```
<h1 id="demo">Lorem ipsum dolor sit amet consectetur adipisicing elit.
<a href="#" target="_blank" style="display: none;">Link</a> ipsum maiores inventore eveniet!</h1>
```

JS

```
let headingOne = document.getElementById('demo')

console.log(headingOne.innerText)

/*
 Output:

'Lorem ipsum dolor sit amet consectetur ad'Lorem ipsum dolor sit amet consectetur adipisicing elit. ipsum maiores inventore eveniet!'

*/
```

3. document.textContent

- Returns the **raw text content of all elements, including hidden text and spacing**.
- It ignores styling and does not trigger a layout calculation. It is highly secure against script injection attacks

HTML

```
 <h1 id="demo">Lorem ipsum dolor sit amet consectetur adipisicing elit.
<a href="#" target="_blank" style="display: none;">Link</a> ipsum maiores inventore eveniet!</h1>
```

JS

```
let headingOne = document.getElementById('demo')

console.log(headingOne.textContent)

/*
 Output:

'Lorem ipsum dolor sit 'Lorem ipsum dolor sit amet consectetur adipisicing elit. Link ipsum maiores inventore eveniet!'

*/
```

Made with GAMMA

Property to change CSS Style

- We can change style of an HTML element using .**style** property in JavaScript.
- **Eg :**

HTML

```
<button class="btn">Submit</button>
```



JS

```
submitBtn = document.querySelector('.btn')

submitBtn.style.backgroundColor = 'black'
submitBtn.style.color = 'white'
```

Submit

Accessing and Changing Attributes of an elements

1. **getAttribute():**

- Retrieves the current value of a specified attribute on an element
- It returns the value as a **string**. If the attribute does not exist, it returns **null**.
- **Eg**

HTML

```
<h1 id="head" class="imp">Lorem ipsum dolor sit amet.</h1>
```

JS

```
let heading1 = document.getElementById('head')
console.log(heading1);

console.log(heading1.getAttribute('class'));
// will print imp
```

2. setAttribute():

- Adds a new attribute to an element or updates the value of an existing one.
- It requires two arguments: the name of the attribute and the value you want to assign to it. If the value is not a string, JavaScript automatically converts it to one
- **Eg**

HTML

```
<h1 id="head" class="imp">Lorem ipsum dolor sit amet.</h1>
```

JS

```
let heading1 = document.getElementById('head')
console.log(heading1);

heading1.setAttribute('class','new imp')
console.log(heading1.getAttribute('class'));
// will print new imp
```

Made with GAMMA

Handling Nodelist and HTMLCollection

1. **Handling NodeList** : We get nodelist from **document.querySelectorAll()**

HTML

```
<h2 id="heading-two">Feeling Hungry ? <a href="#" style="display: none;">Order Now</a> from our partner restaurants</h2>

<h2>Hello My Love</h2>
<h2>Hello My Pet</h2>
<h2>Hello My Mom</h2>
```

JS

```
let bunchOfNodeList = document.querySelectorAll('h2')


bunchOfNodeList.forEach((item)=>{
    item.style.color = 'red'
})
```

2. Handling HTMLCollection : We get HTML collection from **document.getElementByTagName()** and **document.getElementByClassName()** we've to convert HTML Collection into Array using **Array.from()**

HTML

```
<button class="btn">Login</button>
<button class="btn">Signup</button>
```

JS

```
let collectionHTML = document.getElementsByClassName('btn')

let arrCollection = Array.from(collectionHTML)
console.log(arrCollection);

arrCollection.forEach((item) =>{
    item.style.backgroundColor = 'green'
    item.style.color = 'white'
    item.style.padding = '0.7rem'
    item.style.margin = '0.2rem'
    item.style.backgroundColor = 'green'
    item.style.border = 'none'
    item.style.borderRadius = '10px'
})
```

Made with GAMMA

DOM Continuation

Lec-11

Made with GAMMA

Creating New HTML elements using DOM

- To add a **new element to the HTML DOM**, you must **create the element (element node) first**, and **then append it to an existing element**.

1. document.createElement()

- This method generates a new HTML element specified by its tag name. When you call this, the element **only exists in the computer's memory—it is not yet visible on the screen.**

Eg :

```
const heading1 = document.createElement('h1')
console.log(heading1);

// Output: <h1></h1>
```

2. document.createTextNode()

- This method creates a plain text node. It is a safe way to handle text because it automatically escapes any HTML tags, preventing Cross-Site Scripting (XSS) vulnerabilities. Like `createElement()`, t**his node only exists in memory until attached.**

Eg :

```
const h1TextNode = document.createTextNode('Hello world')
```

3. appendChild()

- This method takes a node and adds it to the **end of the children list of a specified parent element. You use this to link your elements together in memory, and finally to hook them to the visible webpage layout** (like appending to `document.body`).

```
const h1TextNode = document.createTextNode('Hello world')
heading1.appendChild(h1TextNode)
document.body.appendChild(heading1)

// Output: <h1>Hello world</h1>
```

Hello world

Made with GAMMA

Creating New HTML elements using DOM

4. insertBefore()

- The **insertBefore()** method in the JavaScript DOM is **used to insert a new node before an existing child node inside a specified parent element.**
- While **appendChild()** always adds an element to the very end, **insertBefore()** gives you precise control over the exact placement.

HTML

```
<body>
    <ul>
      <li>Egg</li>
      <li>Milk</li>
      <li>Bread</li>
    </ul>
</body>
```

Adding a new li as Coffe before milk using JS DOM

```
// Creating new li
let listItem = document.createElement('li')

// Creating Text node for li
let coffee = document.createTextNode('Coffee')

// appending text node to li
listItem.appendChild(coffee)
console.log(listItem);

// selecting the element before which we'll be adding our new element
let list = document.querySelector('li:nth-child(2)')
console.log(list);

// selecting the parent element in which siblings exists
let ul = document.querySelector('ul')

// inserting our newly created element
ul.insertBefore(listItem,list)
```

Made with GAMMA

Editing HTML Elements using DOM

5. replaceWith()

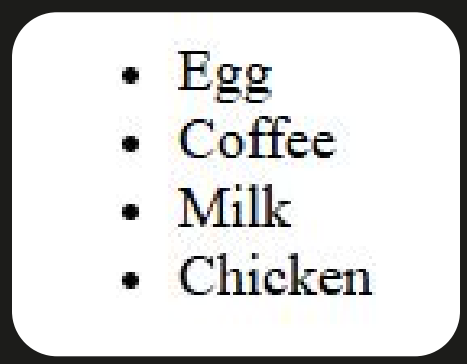
- The `replaceWith()` method in the JavaScript DOM is **used to replace an existing element with one or more new elements or text nodes**.
- Suppose you have the following list and you wish to replace **Bread** with **Chicken.**




```
// selecting element which you want to replace
let itemTobeReplaced = document.querySelector('li:last-child')
console.log(itemTobeReplaced);

// creating and element which will be replacing the older element
let replacingElement = document.createElement('li')
replacingElement.textContent = 'Chicken'

// replacing the old element with new one using replaceWith()
itemTobeReplaced.replaceWith(replacingElement)
```



Made with GAMMA

Remove HTML Element using DOM

6. remove() : The `remove()` method in the JavaScript DOM is **used to completely delete an element from the web page**

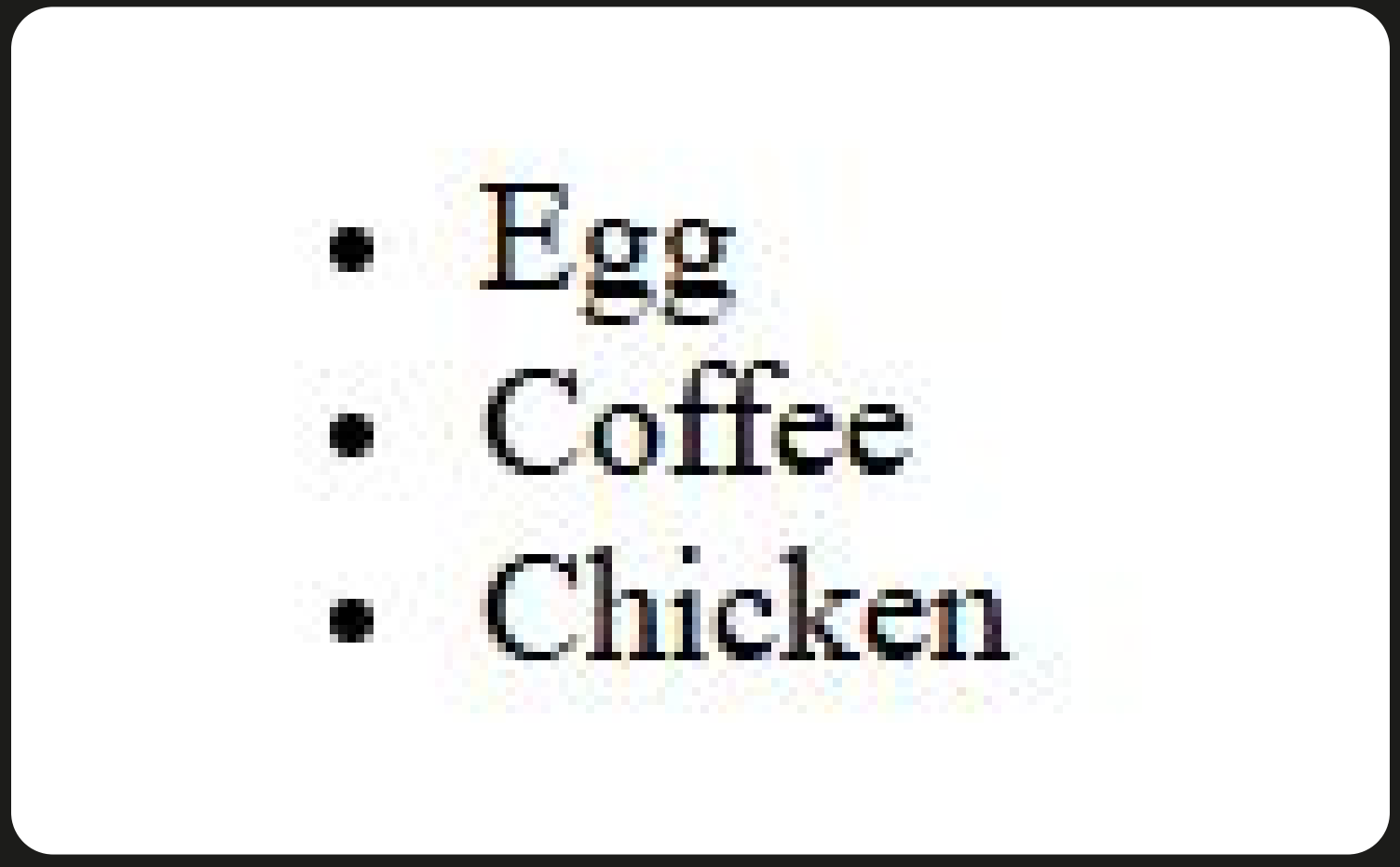
suppose we want to remove **Milk** from our list,



We can remove it in following way:

```
// selecting item to be removed
let itemToBeRemoved = document.querySelector('li:nth-child(3)')
console.log(itemToBeRemoved);

// removing the item using .remove()
temToBeRemoved.remove()
```



Made with GAMMA

Events In JS

Lec-12

Made with GAMMA

Events in JS

- Events are the **things which happen** to HTML elements.
- Eg of events:

1. **An HTML button is clicked**
2. **A web page has finished loading**
3. **The mouse moves over an element**
4. **A keyboard key is pressed**
5. **An HTML input field is changed**

- HTML DOM allows JavaScript to react to HTML events.
- Often when events happen, you may want to do something.
- When **JavaScript is used in HTML pages**, **JavaScript can react on events**.
- **JavaScript lets you execute code when events are detected**.
- HTML allows **event handler attributes**, **with JavaScript code**, to be added to HTML elements.
- In the following example, an **onclick** attribute (with event handler **displayDate()** as it's value), is added to a **<button>** element

Eg:

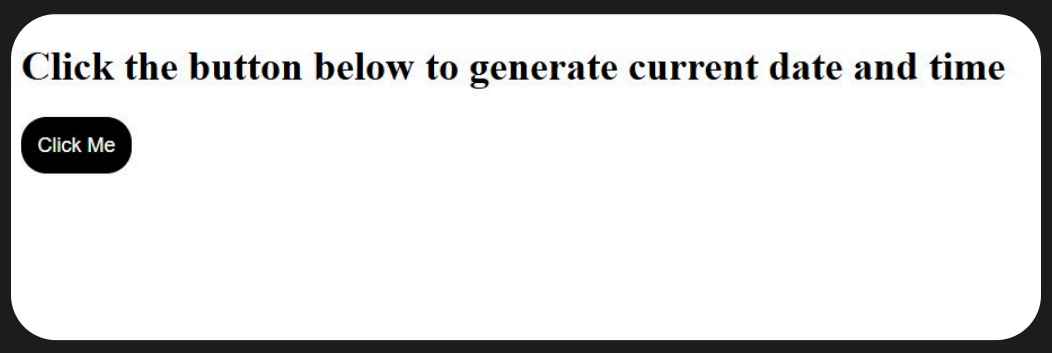
HTML

```
<h1 id="demo">Click the button below to generate current date and time</h1>

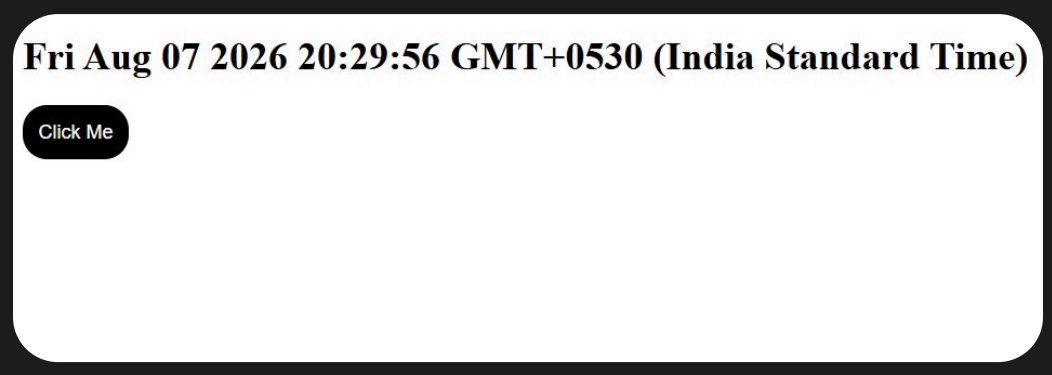
<button onclick="displayDate()">Click Me</button>
```

JS

```
// Event handler
const displayDate = () =>{
  let headingElement1 = document.getElementById('demo')
  headingElement1.innerText = Date()
}
```



Before clicking the button



After clicking the button

Made with GAMMA

Event Listeners

- Using event attributes like **onclick** which we saw in above example are easy to use, **but such way is not recommended**.
- It is recomended to use **addEventListener()** method to handle the events.

addEventListener()

- The **addEventListener()** method attaches an event handler to an specific element without overwriting existing event handlers.
- We can add **many event handlers to one element**, also we can add **many event handlers of same type** to one element.
- When using the **addEventListener()** method, the **JavaScript is separated from the HTML markup**, for better readability and **allows you to add event listeners even when you do not control the HTML markup**.
- **Syntax** :

```
target.addEventListener(eventName, listenerF, true/false)
```

where, **eventName** : name of the event you want to listen for, eg 'click', 'keydown', etc.

listenerFn : Call back function which runs when event fires, this function has **event object (e)** as default argument.

true: capturing phase

false/omitted : bubbling phase

- You can easily remove an event listener by using the **removeEventListener()** method.
- Same Eg which we saw above is rewritten using event listener in following way:

HTML

```
<h1 id="demo">Click the button
below to generate current date
and time</h1>

<button>Click Me</button>
```

JS

```
let headingElement1 = document.getElementById('demo')

document.querySelector('button').addEventListener("click",() =>{
      headingElement1.innerText = Date()
})
```

Made with GAMMA

Event Propagation

- Event propagation is the **order in which an event travels through the DOM** when it happens on a **nested element**.
- Think of it like a pebble dropped in a pond — the click doesn't just happen and stop; it moves through the DOM tree in a defined path.
- There are three phases, in this order:

1. **Capturing phase** — the event starts at the window, travels down through document, then down through every ancestor, until it reaches the actual element that was clicked.
2. **Target phase** — the event reaches the exact element that triggered it.
3. **Bubbling phase** — the event then travels back up, from the target element through all its ancestors, all the way to window.

- By default, **addEventListener** listens during the **bubbling** phase
- **Eg of Bubbling**

HTML

```
<div id="grandparent">
  <div id="parent">
    button id="child-btn">Click it</button>
  </div>
```

JS

```
let grandparent =
document.getElementById("grandparent");

let parent = document.getElementById("parent");

let childBtn = document.getElementById("child-btn");

grandparent.addEventListener("click", () => {
    console.log("grandparent clicked");
 });

parent.addEventListener("click", () => {
    console.log("parent clicked");
});

childBtn.addEventListener("click", () => {
    console.log("childBtn clicked");
});
```

Output:

childBtn clicked

parent clicked

grandparent clicked

Made with GAMMA

Event Propagation

- **Eg of Capturing**

HTML

```
<div id="grandparent">
  <div id="parent">
     <button id="child-btn">Click it</button>
  </div>
</div>
```

JS

```
let grandparent =
document.getElementById("grandparent");

let parent = document.getElementById("parent");

let childBtn = document.getElementById("child-btn");

grandparent.addEventListener("click", () => {
      console.log("grandparent clicked");
}, true);

parent.addEventListener("click", () => {
      console.log("parent clicked");
 },true);

childBtn.addEventListener("click", () => {
      console.log("childBtn clicked");
}, true);
```

Output:

grandparent clicked

parent clicked

childBtn clicked

Made with GAMMA

Mouse Events in JS

1. **click :** Fires when an element is pressed and released with the primary mouse button (or tapped on touch devices).

HTML

```
<button id="btn">Click Now</button>
```

JS

```
let clickNowBtn = document.getElementById("btn");

clickNowBtn.addEventListener("click", function (e) {
clickNowBtn.innerText = "Button Clicked";
 });
```

before clicking



after clicking



2. dblclick : Fires when an element is clicked twice in quick succession

HTML

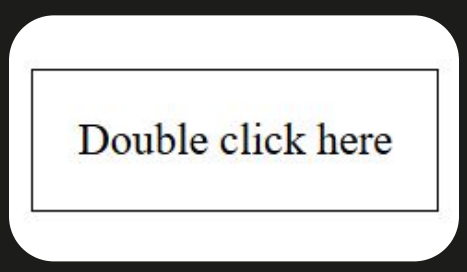
```
<div id="box">Double click here</div>
```

JS

```
let box = document.getElementById("box");

box.addEventListener("dblclick", () => {
      box.innerHTML = "Event triggered";
   });
```

before double clicking



after double clicking



Made with GAMMA

Mouse events in JS

3. mousedown : Fires the moment a mouse button is pressed down over an element (before release).

HTML

```
<div id="area">Press Down Here</div>
```

JS

```
let area = document.getElementById("area");

area.addEventListener("mousedown", () => {
    area.innerHTML = "Event triggered";
  });
```

4. mouseup : Fires when a mouse button is released over an element (after release).

HTML

```
<div id="area">Press Down Here</div>
```

JS

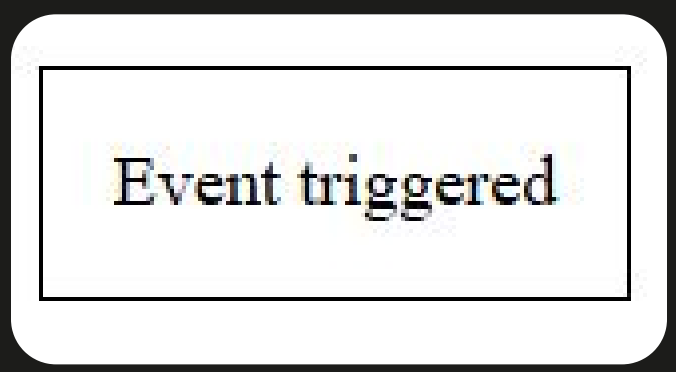
```
let area = document.getElementById("area");

area.addEventListener("mouseup", () => {
    area.innerHTML = "Back to normal";
  });
```

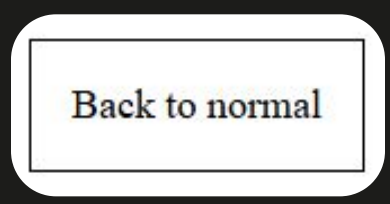
before event triggering



mousedown



mouseup



Made with GAMMA

Mouse Events in JS

5. mouseover: Fires when the cursor moves onto an element or any of its children.

HTML

```
<div class="card-parent">
  <div class="card">Bring your mouse here</div>
</div>
```

JS

```
let card= document.querySelector('.card')
card.addEventListener('mouseover',()=>
{
  document.querySelector('.card').innerText = 'Mouse
over'
})
```

6. mouseout: Fires when the cursor leaves an element or moves onto one of its children.

HTML

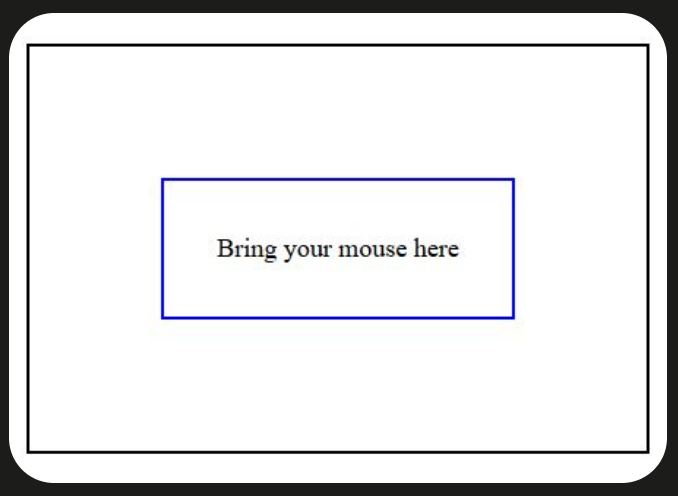
```
<div class="card-parent">
  <div class="card">Bring your mouse here</div>
</div>
```

JS

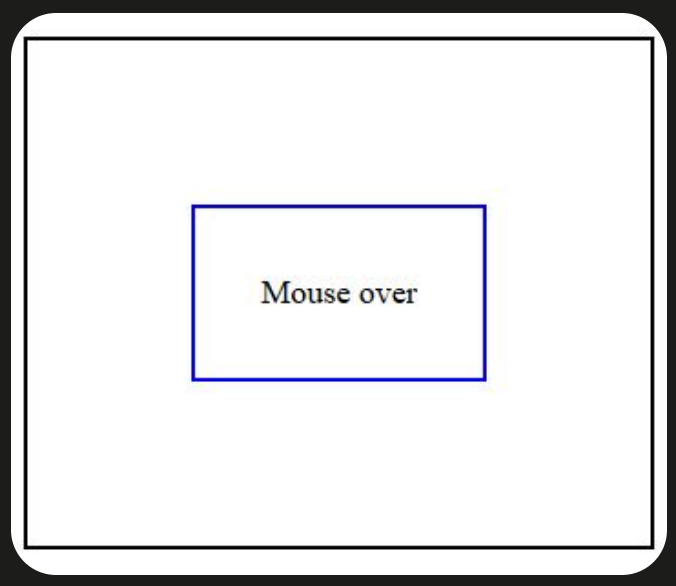
```
let card = document.querySelector('.card')

card.addEventListener('mouseout',()=>
{
  document.querySelector('.card').innerText = 'Mouse
out'
})
```

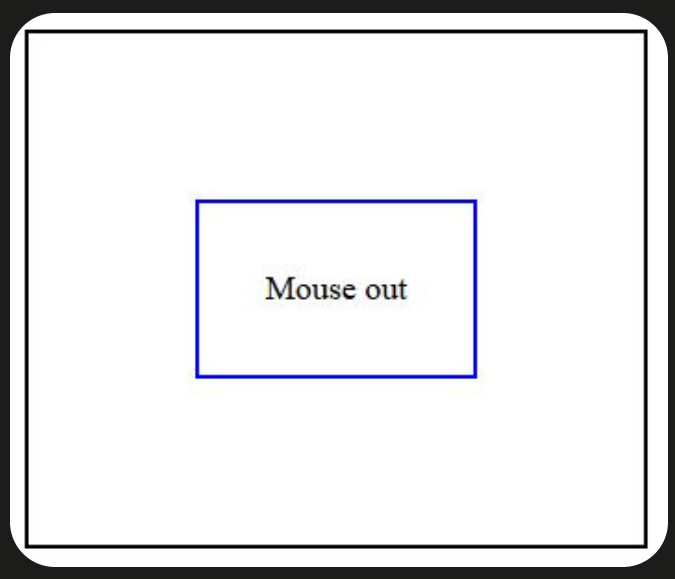
before event trigger



mouseover



mouseout



Made with GAMMA

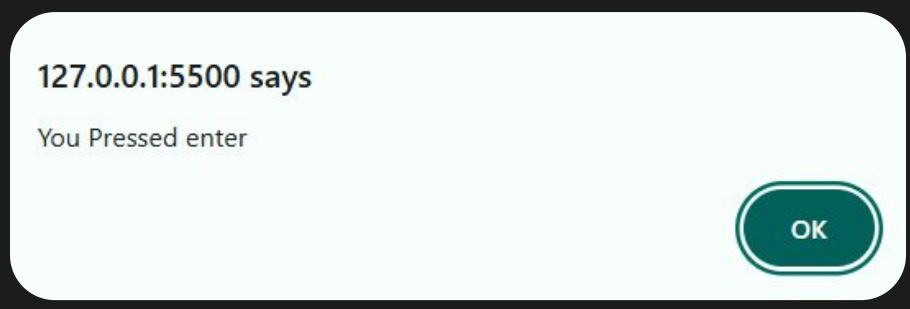
Keyboard Events in JS

1. **keydown:** Fires when a key is pressed down. Repeats continuously if the key is held.

JS

```
document.querySelector('body').addEventListener('keydown',
(e)=>
{
    if(e.key === 'Enter'){
      alert('You Pressed enter')
}
```

Output after pressing the key

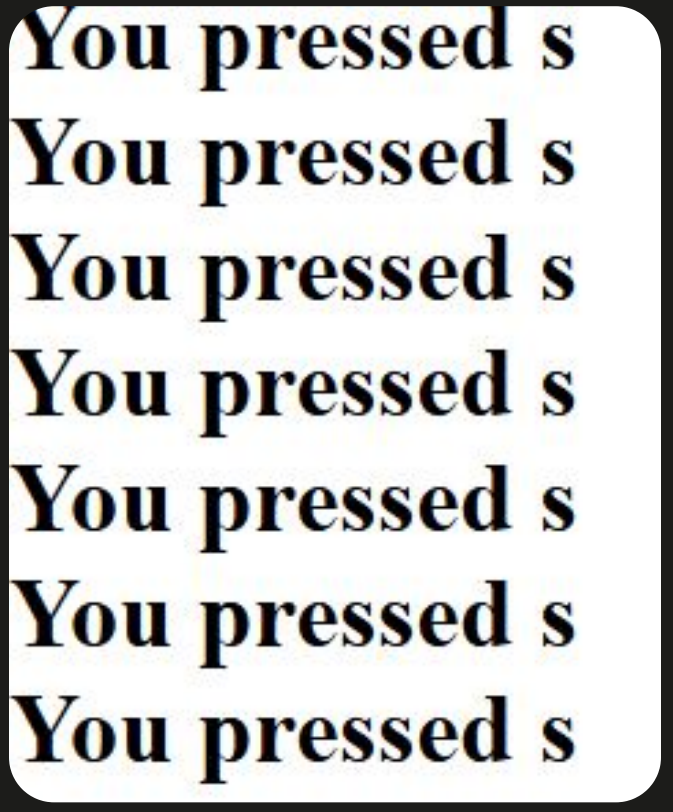


2. keyup: Fires once when a key is released.

JS

```
document.querySelector('body').addEventListener('keyu
p',(e)=>
{
 if(e.key === 's')
 {
   let newElem = document.createElement('h1')
   let newText = document.createTextNode(`You pressed
${e.key}`)
   let brElem = document.createElement('br')
   newElem.appendChild(newText)
   newElem.appendChild(brElem)
   document.body.appendChild(newElem)
   }
})
```

Output after pressing key



Self Study

- Learn more about **Form Events, Window & Document Events, Drag & Drop Events, preventDefault**

Made with GAMMA

Browser Object Model (BOM)

The **Browser Object Model (BOM)** is used to **interact with the browser**.

Window Object

- The window object **represents a window in browser**.
- An window object is **created automatically by the browser** and is a **default object of the browser**.
- All global JavaScript **objects**, **functions**, and **variables with the var keyword** automatically **becomes member of the window object**.
- **Variables with let** and **const** keyword created globally **does not get attached to the window object**, they **live in script scope** yet they are considered global.
- **Global variables** are **properties of the window object**.
- **Global functions** are **methods of the window object**.

Made with GAMMA

Asynchronous JavaScript

Lec-13

Made with GAMMA

How JS Works Behind the Scene

- **JavaScript Execution Context** : It is an abstract environment created by JavaScript Engine, to execute and evaluate JavaScript code.
- In JavaScript Execution context are of two types:

1. **Global Execution Context (GEC)** : Creates automatically by the JS engine when your script runs first. **Except the code inside any function** GEC **handles all basic level code** and **creates global object (eg. window object in browsers)**. There is only one GEC per script execution.
2. **Function Execution Context (FEC)** : Get's created dynamically everytime a function is invoked. Each function call get's its **own independent context** to execute it's local code.

- Whenever JavaScript Execution Context is generated, it runs in two different phases:

1. **Memory Creation Phase**
2. **Execution Phase**

Let's try and breakdown how below given code will execute:

Step-1 : **Global Execution Context (GEC)** get's created and get's allocated to 'this'.

Step-2 : **Memory Creation Phase** get's created, in which all the variable get's collected and stored in it.

val1 ← undefined

val2 ← undefined

addVal← fn definition

res1 ← undefined

res2 ← undefined

This concludes our Memory creation phase

Step-3 : **Execution phase** starts

val1 ← 10

val1 ← 20

addVal ← it will **create it's own execution context**, in which **new variable environment** and **Execution thread** get's generated



since new sandbox is now created for **addVal()**, **it would again create Memory Creation Phase and Execution Phase** for this new sandbox.

Step- 3.1 : **Memory creation phase** of addVal()

num ← undefined

num 1← undefined

total ← undefined

Step- 3.2 : Execution phase of addVal()

num ← 10

num1 ← 20

total ← 30

After this new execution Context created for addVal() get's destroyed.

Step-3 Continues:

res1 ← 30

Made with GAMMA

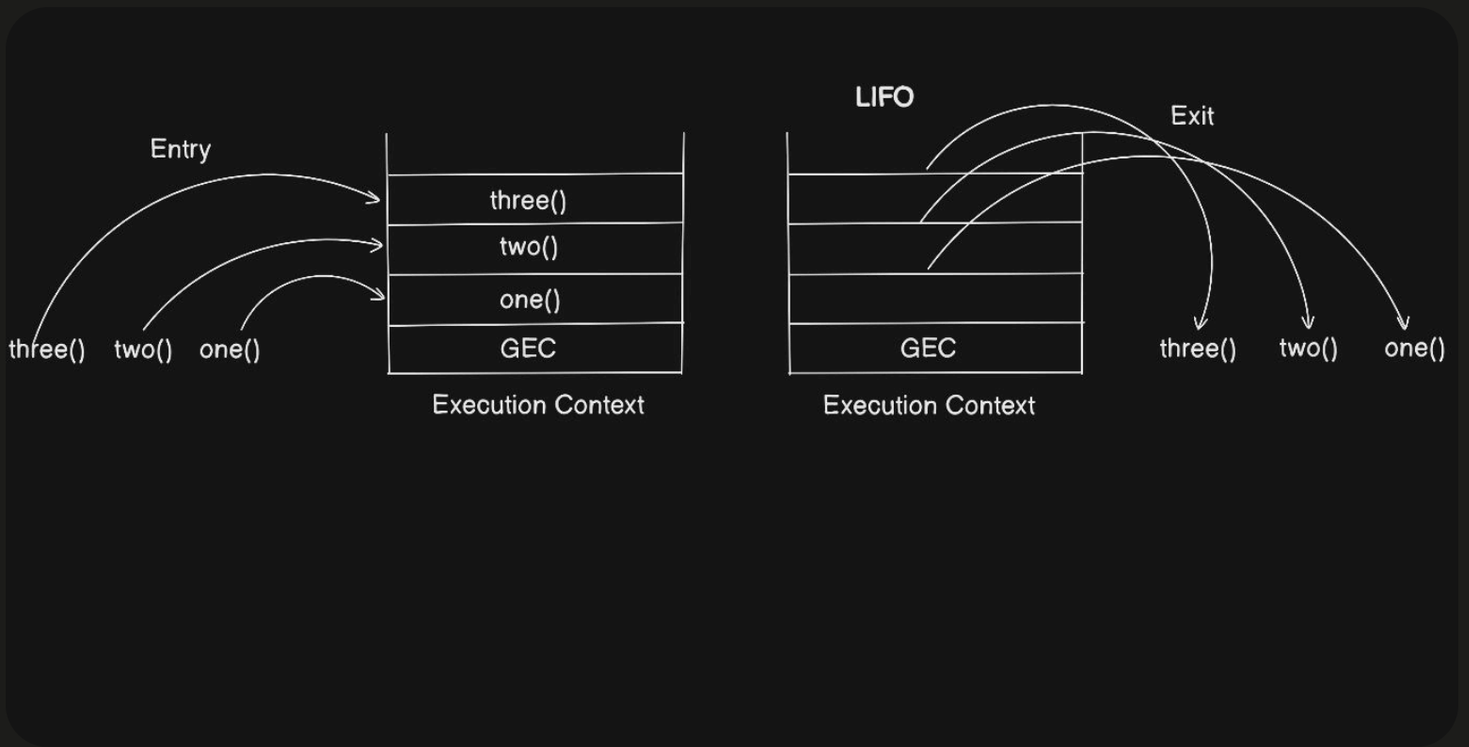
Call Stack

- The call stack is a data structure that JavaScript uses to keep track of function call and execution contexts in a program.
- It operates on a **LIFO (Last-In, First-Out)** basis, so the function called last finishes first.
- Since, JavaScript is **single-threaded**, meaning it **has only one call stack and can execute only one thing at a time**.
- The stack remembers which function is running and where to return after it finishes.

Eg:

```
// Call Stack
function one(){
    console.log("one");
    two()
}
function two(){
    console.log("two");
    three()
}
function three(){
    console.log("three");
}

one()
two()
three()
```



Made with GAMMA

What is Asynchronous JavaScript ?

- Generally, **JavaScript runs one task at a time**.
- While **one task is running**, **no other JavaScript code can run**.
- If a task takes a long time, the browser cannot respond to user actions until the task is finished.
- Asynchronous programming solves this problem **by allowing long-running operations to complete in the background while JavaScript continues running other code**.
- The word **"async"** means - **"not at same time"**.
- Instead of waiting for one task to finish before starting the next, JavaScript can continue running other code while waiting for an operation to complete.
- When the operation finishes, JavaScript continues by running code that handles the result.

Why Asynchronous Programming ?

- Some operations take time to complete like- **downloading files, waiting for timers, waiting for events** or **waiting for network requests**, etc.

Timers	Runs after specified milliseconds
Events	Runs when triggered by an event
Network requests	Runs when data arrives.

- If JavaScript waited for these tasks, the page would freeze, **a frozen page is an broken page**.
- To stay **responsive**, JavaScript uses **async programming.**
- **Asynchronous flow** refers to **how JavaScript allows certain operations to run in the background and let their results be handled when they are ready.**

Made with GAMMA

Asynchronous Timer Functions

- JavaScript timers let you run a function **after a delay** or repeatedly **at fixed intervals**.
- **Timer functions do not pause the running JavaScript**.
- JavaScript immediately continous with the next statement.
- The two main timer functions are **setTimeout()** and **setInterval()**

1. setTimeout()

- The **setTimeout()** function **schedules a function to run once after a delay**.
- It is an **Higher Order Function**, as it **takes an callback function as one of it's arguments**.
- **Syntax**:

```
setTimeout(function, milliseconds)
```

- In above syntax, **first argument** is the **callback function** to run.
- The **second argument** is the **delay in milliseconds**.
- **1 sec = 1000 milliseconds**
- **Eg:**

```
console.log("Wait for a message after 2 sec");

 // setTimeout()
 setTimeout(()=>{
     console.log("Hello I am Asynchronous message");
  },2000)
```

- If we add **0 as second argument** for a setTimeout() function, in such case the **callback will execute after the JavaScript has been completely executed**.

Made with GAMMA

Asynchronous Timer Functions

2. clearTimeout()

- The **setTimeout() function returns a timer identifier**.
- We can **pass the timer identifier to clearTimeout() to cancel the setTimeout()**.
- **Syntax**:

```
clearTimeout(timerId)
```

- **Eg:**

```
const startBtn = document.getElementById('start-btn')
const stopBtn = document.getElementById('stop-btn')
let timerDisplay = document.getElementById('timer-display')
let timerBehaviour;

startBtn.addEventListener('click', ()=>{
            let timerBehaviour = setTimeout(()=>{
            timerDisplay.innerText = 'Timer Started after 1 sec'
          },1000)
    })

 stopBtn.addEventListener('click',()=>{
             clearTimeout(timerBehaviour)
             timerDisplay.innerText = 'Timer Stopped'
 })
```



On clcking **'Start Timer'**



On clicking **'Stop Timer'**



Practice Question

You're building an idle-session handler for a web app. **If the user doesn't interact with the page (no mouse move, click, or keypress)** for 10 seconds, show a warning message. If they still don't interact for another 5 seconds after that, log them out automatically. But if the user does anything at all, the whole countdown should reset from zero — the warning should never fire if they're actively using the app.

Write a function **startIdleTimer()** that implements this behavior. Specific requirements:

- **After 10 seconds of inactivity**, log **"Warning: You will be logged out due to inactivity"** to the console.
- **After 15 seconds total of inactivity (5 seconds past the warning)**, log **"User logged out"**.
- Listen for **mousemove, keydown, and click events on the document** — any of these should reset both timers back to zero.
- If the **user keeps interacting continuously, neither message should ever print**.
- Make sure **you're not stacking up timers with every event — each reset should clean up the previous ones properly**.

Made with GAMMA

Asynchronous Timer Functions

3. setInterval()

- The **setInterval()** function **runs a callback function repeatedly**.
- It is also an **Higher Order Functions**.
- The callback function here is called repeately with (at least) specified delay in between.
- **Syntax**:

```
setInterval(function, milliseconds)
```

- **Eg**:

```
setInterval(() => {
  let h3 = document.createElement("h3");
  let thirdHeadingText = document.createTextNode("Hello");
  h3.appendChild(thirdHeadingText);
  document.body.appendChild(h3);
}, 2000);
```

- If we add **0 as second argument** for a setInterval() function, in such case the **callback will execute after the JavaScript has been completely executed.**

Made with GAMMA

Asynchronous Timer Functions

4. clearInterval()

- The **setInterval() function returns a timer identifier**.
- We can **pass the timer identifier to clearInterval() to cancel the setInterval().**
- **Syntax**:

```
clearInterval(intervalId);
```

- **Eg**:

```
let countdownTillZero = setInterval(()=>{
document.body.innerText = `Timer reading
${timeinSec}`
timeinSec--

if(timeinSec < 0){
    clearInterval(countdownTillZero)
    document.body.innerText = 'Timer killed'
}},1000)
```

Timer reading 5

Timer reading 4

Timer killed

Practice Question

You're building an idle-session handler for a web app. **If the user doesn't interact with the page (no mouse move, click, or keypress)** for 10 seconds, show a warning message. If they still don't interact for another 5 seconds after that, log them out automatically. But if the user does anything at all, the whole countdown should reset from zero — the warning should never fire if they're actively using the app.

Write a function **startIdleTimer()** that implements this behavior. Specific requirements:

- **After 10 seconds of inactivity**, log **"Warning: You will be logged out due to inactivity"** to the console.
- **After 15 seconds total of inactivity (5 seconds past the warning)**, log **"User logged out"**.
- Listen for **mousemove, keydown, and click events on the document** — any of these should reset both timers back to zero.
- If the **user keeps interacting continuously, neither message should ever print**.
- Make sure **you're not stacking up timers with every event — each reset should clean up the previous ones properly**.

Made with GAMMA

Promises in JS

Lec-14

Made with GAMMA

Asynchronous Promises

- A Promise represents an **future result of an asynchronous operation**.
- It acts as a placeholder for a value not that is not available yet.
- **A Promise does not make code asynchronous**, It represents the **result of an asynchronous operation.**
- When an operation is finished, **Promise is either fulfilled with a value or rejected with an error**.
- Every promise is always in one of the three states:
    - **Pending**: The Operation has started but not finished.
    - **Fulfilled**: The Operation completed succesfully.
    - **Rejected**: The Opearation failed.



- **A promise is settled when it is fulfilled or rejected**.

Creating a Promise

- We can **create** promise using **promise constructor**
- **Syntax:**

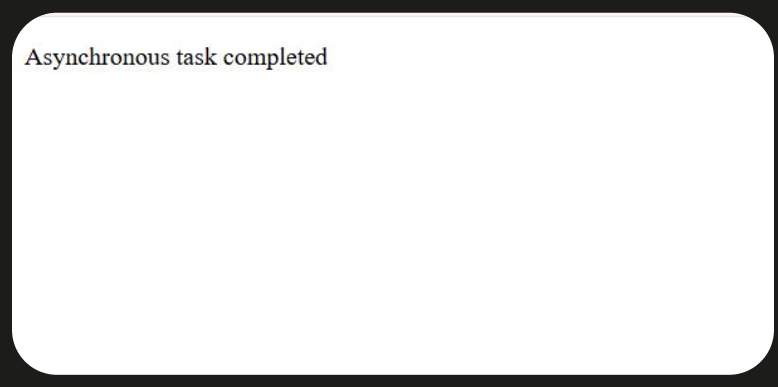
```
const promise = new Promise(function(resolve, reject))
```

- The Constructor **receives a callback function** with two parameters.
- **resolve**: Fulfills the promise with a value.
- **reject**: Rejects the promise with an error.

> • **Note: Call either resolve or reject, a promise can be settled only once**.

- **Eg to create a promise:**

```
const newPromise = new Promise (function(resolve, reject){
  setTimeout(() => {
      document.body.innerHTML = 'Async task completed'
   }, 2000);
})
```



Made with GAMMA

Asynchronous Promises

Using a Promise

.then()

- If the Promise is fulfilled, we can handle the promise using **.then()** method.
- The registered function receives the fulfillment value.
- **Eg to use a promise completely**:

```
newPromise.then(()=>{
      document.body.innerHTML = 'Promise consumed successfully'
})
```

Asynchronous task completed

Promise consumed succesfully

- Inside **resolve(),** we tend to pass a **promise value**, which can be a **single value**, an **object** or an **array**, etc.
- This **promise value** is **passed as a parameter to .then()**
- **Eg**

```
new Promise((resolve, reject)=>{
 setTimeout(() => {
  resolve({
            user_id: "angel_priya",
            password: "priya@146"
          })
    }, 3000);
  }).then((user)=>{
       console.log(user);
 })
```

Output at the console

{user_id: 'angel_priya', password: 'priya@146'}

- If you want to print a particular property of an object which is getting passed in resolve, then this can be achieved by chaining of .then()

```
new Promise((resolve, reject)=>{
  setTimeout(() => {
   resolve({
        user_id: "angel_priya",
        password: "priya@146"
   })
    }, 3000);
   }).then((user)=>{
        return user.user_id;
   }).then((userid)=>{
        console.log(userid);
   })
```

Output at the console will have value of user_id

angel_priya

Made with GAMMA

Asynchronous Promises

.catch()

- The **catch()** method registers a function that handles a rejected Promise
- **Eg**:

```
new Promise((resolve,reject)=>{
    setTimeout(() => {
      let error = false;
       if(!error){
         resolve({
           acc_no: 15324923430,
           ifsc: "BOBBKC"
         })
       }
      else{
          reject("ERROR, Can't fetch details")
           }
}, 4000);
   }).then((user)=>{
         console.log(user);
  }).catch((error)=>{
         console.log(error);
```

Output:

When Promise resolved

{acc_no: 15324923430, ifsc: 'BOBBKC'}

When Promise rejected

ERROR, Can't fetch details

.finally()

- The **finally**() method **registers a function that runs when a Promise settles**.
- It runs whether the Promise is fulfilled or rejected.
- **Eg**:

```
new Promise((resolve,reject)=>{
    setTimeout(() => {
      let error = false;
       if(!error){
         resolve({
           acc_no: 15324923430,
           ifsc: "BOBBKC"
         })
       }
  else{
      reject("ERROR, Can't fetch details")
      }
}, 4000);
   }).then((user)=>{
         console.log(user);
  }).catch((error)=>{
         console.log(error);
```

Output:

{acc_no: 15324923430, ifsc: 'BOBBKC'}

Promise succesfully settled

Made with GAMMA

Async and Await

- JavaScript also provides the **async** and **await** keywords, that **let's you write Promise-based code in a style that looks like synchronous JavaScript**.
- These keywords make Promise-based code easier to read.

Async Keyword

- The **async** keyword before a function **makes a function return a promise**.
- This is true even if you return a normal value.
- If the **function returns a value**, **JavaScript automatically wraps that value in a Promise.**
- **Eg :**

```
async function helloWorld(){
    return 'Hello world'
}
```

Above function **"helloWorld"** is a async function which is returning a promise.

Now, we have to handle the promise return by async function helloWorld.

```
helloWorld().then((value)=>{
    console.log(value);
})
```

After handling a promise by above given code, we would see **"hello world"** inside the console, **as the async function was returning "hello world" as a promise**.

Made with GAMMA

Async and Await

- As we saw in above example that any function when initated by an **async** keyword, **returns a promise**. That promise now is further handled along with the help of **await keyword**.

Await Keyword

- The **await** keyword **waits for an promise to be settled first**.
- It can **only be used inside an async function** or at the **top level of a JavaScript module**.
- While the **async function is waiting,** the rest of the program can continue running.
- The **await** keyword **pauses only the current async function**. **It does not pause the whole JavaScript**.
- Let's take an example of following promise and handle it using async and await.
- **Eg of a promise**

```
const pracPromise = new Promise((resolve, reject) => {
  setTimeout(() => {
    let error = false;

    if (!error) {
      resolve({
        prod_id: 111,
        prod_name: "Mobile cover",
      });
    } else {
      reject("ERROR, Product cannot be shown");
    }
  }, 2000);
})
```

Handling of promise using async and await.

```
// creating a async function
async function handlePracPromise(){
const pracPromiseResponse = await pracPromise
console.log(pracPromiseResponse);
}
// calling an async function
handlePracPromise();
```

Output at the console when no error is in the promise

{prod_id: 111, prod_name: 'Mobile cover'}

a. **prod_id**: 111
b. **prod_name**: "Mobile cover"
c. [[Prototype]]: Object

- In above case, our promise didn't had any error, **but in case of any error we have to handle using try and catch block inside a async function**.

Handling of promise having error using try and catch block in async and await

```
const pracPromise = new Promise((resolve, reject) => {
  setTimeout(() => {
    let error = true;

    if (!error) {
      resolve({
        prod_id: 111,
        prod_name: "Mobile cover",
      });
    } else {
      reject("ERROR, Product cannot be shown");
    }
  }, 2000);
})
```

```
async function handlePracPromise() {
 try {
const pracPromiseResponse = await pracPromise;
console.log(pracPromiseResponse.prod_name);
} catch (err) {
    console.log(err);
  }
}
```

Output from the console

ERROR, Product cannot be shown

Made with GAMMA

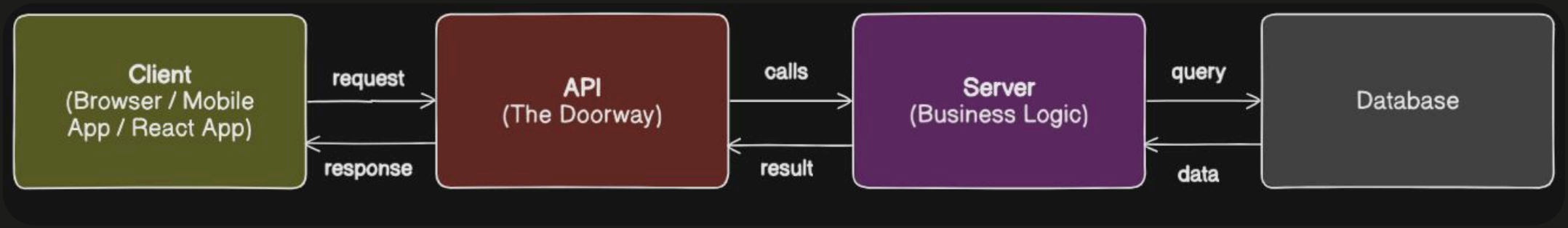
Fetch API in JavaScript

Lec-15

Made with GAMMA

What is an API

- **API** Stands for- **Application Programming Interface**.
- It is a **defined way for one piece of software to ask another piece of software to do something** (or to give it some data) and to get an answer back, **without needing to know how that other software works inside**.
- **Application**: Any software that does a job: a weather service, a payment system, your own backend.
- **Programming**: The API is used by programs (code written by developers), not by end users clicking buttons.
- **Interface**: The meeting point between two systems. Like a contract: "if you send me this, I promise to reply with that."
- **Client only talks to the API**, it never touches the server internals or database directly.



Made with GAMMA

Types of API

- API is categorised into **two** types, **based on who can use them** and **based on style/protocol**.

Category-1: By who can use them

Type	Who can access	Example
Public / Open	Any Developer (often needs a free API key)	OpenWeather, PokeAPI, GitHub public API
Partner	Selected business partners under agreement	A bank exposing an API to approved fintech apps.
Private / Internal	Only inside one company	Your comapny's backend used by it's own web and mobile app
Composite	Combines severak API calls into one.	One endpoint which fetches user + order + payment together.

Category-2: Based on Style/ Protocol

Style	What it is	Data Format	Typical use
REST	Resources identified by URL's, manipulated with HTTP methods. The most common style.	JSON (mostly)	Almost every web/mobile backend.
GraphQL	One endpoint; the client writes a query describing exactly which field it wants.	JSON	Complex UI's, mobile apps needing fewer round trips.
SOAP	Older, strict, XML-based protocol with formal contracts (WSDL).	XML	Legacy enterprise, banking, telecom systems
gRPC	High-performance remote procedure calls over HTTP/2 using Protocol Buffers.	Binary (protobuf)	Service-to-service calls inside microservices.
WebSocket	A persistent two-way connection; both sides can send anytime.	Any (often JSON)	Chat, live dashboards, multiplayer games.
Webhook	Server calls your URL when an event happens ("reverse API").	JSON	Payment confirmations, GitHub push events.

Made with GAMMA

HTTP Methods

- The **method tells the server what action you want to perform on the resource in the URL**.

Method	Purpose
GET	Read/Fetch data
POST	Create a new resource, or trigger an action
PUT	Replace a resource completely
PATCH	Update only some fields
DELETE	Remove a resource
HEAD	Same as GET, but only headers, no body.
OPTIONS	Ask which methods are allowed (CORS preflight)

Made with GAMMA

HTTP Status Codes

- Every response starts with a three-digit status code so the client knows immediately what happened, before reading the body.
- **The first digit tells the category**



Code	Name	When to use it
200	OK	Request succeeded (GET, PUT, PATCH). Body contains the result.
201	Created	A new resource was created (after POST), Add a ***Location*** header.
204	No Content	Success, nothing to return (commonly after DELETE).
301 / 302	Moved Permanently / Found	Redirect to another URL.
304	Not Modified	Cached copy is still valid; no body sent.
400	Bad Request	Malformed request or invalid input (bad JSON, missing field).
401	Unauthorized	**Not authenticated**: no token, bad token or expired token. "Who are you?"
403	Forbidden	**Authenticated but not allowed.** "I know you, but you cannot do this."
404	Not Found	Resource does not exist.
405	Method Not Allowed	URL exists but not for this method (eg. DELETE on a read-only-route)
409	Conflict	Conflicts with current state, eg email already registered.
422	Unprocessable Entity	JSON is well-formed but fails validation rules.
429	Too Many Requests	Rate limit exceeded. Check the ***Retry-After*** header.
500	Internal Server Error	Unexpected bug on the server.
502 / 503 / 504	Bad Gateway/ Unavailable / Gateway Timeout	Upstream service failed, server overloaded or down, or upstream too slow.

Made with GAMMA

Fetch API

- The **Fetch API**, lets JavaScript **download resources** from a web server.
- It **provides an interface for fetching (sending/receiving) resources**.
- It can be **used to load text files, JSON data, images** and many other type of resources.
- The **fetch() method is used to fetch a resource (data)**.
- The **fetch() method returns a promise and that resolves to a response object**.
- **Syntax**:

```
let promise = fetch(url,[options])
```

- **Eg: Suppose we're fetching a response from the following api url**

```
let apiLink = "https://randomuser.me/api/"
```

As we know **fetch()** method will return a promise, so in order to handle it we will use async await inside a try and catch block.

```
const sportsApiReq = async ()=>
{
 try {
     console.log('Getting Data...');
     let response = await fetch(apiLink)
     console.log(response);
    }
 catch (error)
 {
    console.log(error);
 }
}
```

```
Getting Data...                                    fetch-demo.html:22
                                                   fetch-demo.html:24
▸ Response {type: 'cors', url:
  'https://randomuser.me/api/', redirected: false, statu
  s: 200, ok: true, …}
>
```

The response object can have following properties/ methods:

Property/ Method	Description
ok	True if request succeded
status	The HTTP status code
statustext	The HTTP status message
text()	Reads the response as text
json()	Reads the response as JSON
blob()	Reads the response as binary
bytes()	Reads the response as bytes
arrayBuffer()	Reads the response as an Array buffer

Promise.all()

- Used to fetch data from multiple sources at once.

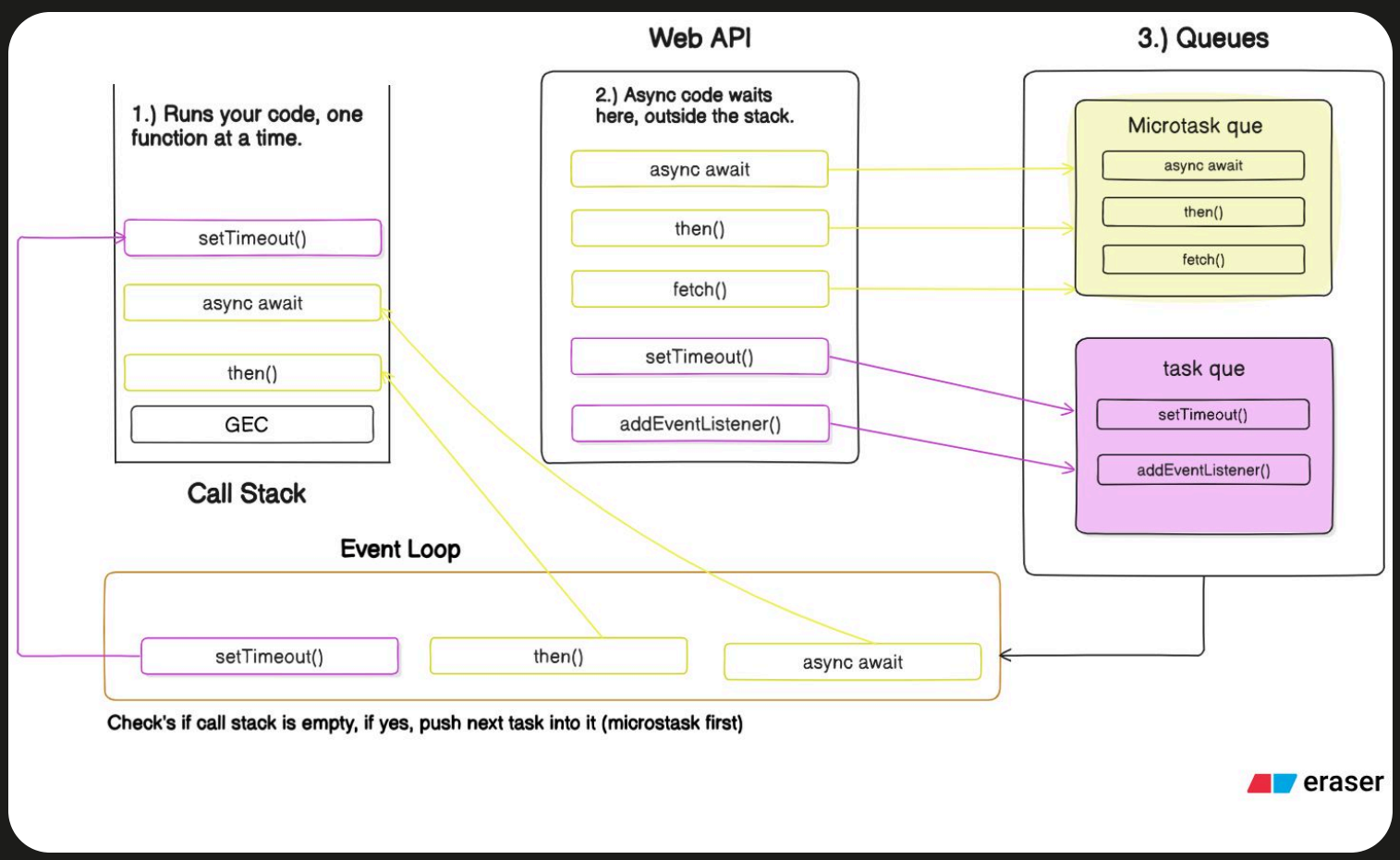
```
async function userAndCountry(){
  try{
     const [userResponse, countryResponse] = await Promise.all([fetch(apiLink1), fetch(apiLink2)])
     const JsonUserResponse = await userResponse.json()
     const JsonCountryResponse = await countryResponse.json()

     console.log(JsonUserResponse);
     console.log(JsonCountryResponse);
    }catch(error){
          console.log(error);
       }
 }
userAndCountry()
```

Made with GAMMA

Event Loop

- The event loop **keeps checking whether the call stack is empty or not**.
- If it is empty, **it takes the next waiting callback from a queue and pushes it onto the stack**.
- The working of event loop are as follows:



Let's understand the whole diagram in 4 simple steps:

Step-1 - Run on the call stack : Your code executes one function at a time. Synchronous code, like console.log(1), runs straight away.

Step-2 - Run on the call stack : Calls like **setTimeout, fetch and click handlers are given to the browser**, which **waits on them outside the stack**, so nothing blocks.

Step-3 - Queue the callback when it's ready : Promise **callbacks (.then) go into the microtask queue**. **Timer and event callbacks go into the task queue**.

Step-4 - The event loop moves callbacks to the stack : When the **call stack is empty**, the **loop pushes the next waiting callback onto it**, always emptying the microtask queue before taking the next task.

Made with GAMMA